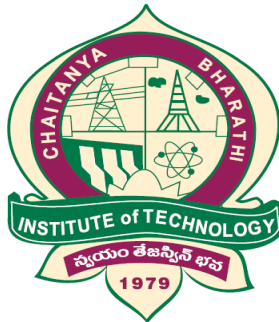


CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY (A)
Gandipet, Hyderabad-500075



LABORATORY MANUAL

CASE STUDY LAB

Syllabus Ref. No: **18CSC27**

B.E VI SEM
Undergraduate Course
in
COMPUTER SCIENCE AND ENGINEERING

Prepared by: T.SUVARNA KUMARI
Asst. Prof., CSE DEPT.



CHAITANYA BHARATHI
INSTITUTE OF TECHNOLOGY (A)
Affiliated to Osmania University

Department of Computer Science and Engineering

Vision:

To become a center of excellence in the field of Computer Science and Engineering that produces innovative, skillful and socially responsible professionals who can contribute significantly to industry and research.

Mission:

The mission of Computer Science and Engineering Department is to:

1. Identifying emerging areas in the field of Computer Science & Engineering and incorporating in the curriculum
2. Providing professionals to the country with innovative ideas in certain areas of advanced computing technologies through research and graduate studies
3. Participation in the design and developmental process of industries and society.

Program Education Objectives (PEOs):

After the completion of the program, our:

1. Practice their profession with confidence by applying new ideas and technologies for the sustainable growth of Industry & Society.
2. To pursue higher studies for professional growth with superior ethics & Character.
3. Engage in Research leading to innovations/products or become a successful Entrepreneur.

Program Specific Outcomes (PSOs):

At the end of the program

PSO1: Machine Learning and Computer Vision: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments

PSO 2: Open source Technologies: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

18CSC27**CASE STUDY**

Instruction	2 Hours per week
Duration of End Examination	2 Hours
Semester End Examination	35 Marks
Continuous Internal Evaluation	15 Marks
Credits	1

Case studies are common in engineering where we analyse (study) situations. Case study exercise is a realistic simulation of a real life situation or strategic problem we are likely to encounter in our workplace or surroundings. A case study is actually “*analysing, applying engineering and science knowledge, reasoning and drawing conclusions*” to solve a real situation. Case studies are different types including historical, real life, problem oriented etc.

Course Objectives: The objectives of this course are

1. To expose students to real life problems/events/situations and technologies
2. To promote individual study, critical thinking and group discussions to build team work
3. To inculcate the culture of self-learning, professional ethics communication

Course Outcomes: On successful completion of the case study, students will be able to

1. Understand real life situations, problems, developments of technologies in Computer science
2. Interpret, analyse, and think critically about the events, situations and gather information from various sources for formulating solutions
3. Apply learned knowledge and commit to decisions
4. Evaluate the approach and solution to the event/problem by considering efficiency and optimization
5. Communicate efficiently both in written and orally to discuss the recommendations

Suggestions to select case studies

- For a *real situation case study*, you can choose an event at your workplace to analyse.
- For a *historical case study*, you can take a recent collapse/development of a company /technology /project (Cambridge Analytica, Google, Facebook, AI, ML, IoT, GitHub, GNU, LibreOffice, FOSS etc.) and analyse what went wrong or gave raise.
- For a *problem oriented case study*, choose a problem where they need to (Situation-- Problem-Solution(s)--Evaluation):
 - understand the situation faced (significance),
 - analyse the specific problem to be tackled,
 - create, analyse, and refine a solution and
 - further evaluate, improve and implement

Instructions:

- Students need to choose a case in consultation with any one of their class teachers and mentor
- The topic to be confined to the areas/courses already completed.
- Submit an abstract consisting of the significance, objectives, methodology and work plan by the end of 3rd week
- Every week they need to show progress to the concerned teacher and mentor
- Shall *present/demonstrate* and submit a *report*(read the Case Study guide lines)

Assessment: The main focus of case studies are to assess the *approach* and the *solution* arrived. In fact, case studies are usually designed not to have one ‘correct’ answer. As long as the students justify their recommendation, and stand up to interrogation from the assessor, they are likely to score marks. Students will be monitored by an internal teacher along with their mentors and evaluated by the external examiner at end.

INDEX

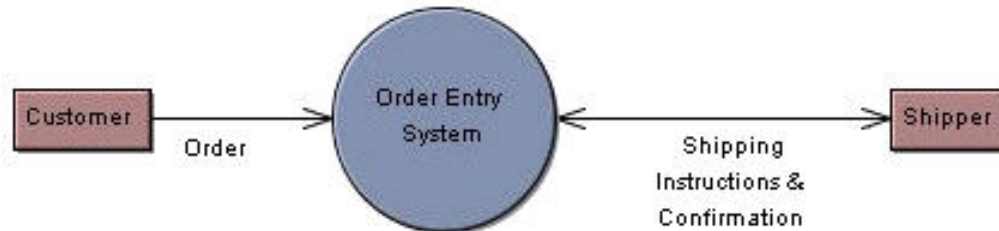
Data Flow Diagram.....	3
Entity Relationship Diagram.....	8
Functional Decomposition.....	13
Dynamic Model Using Finite State Machine.....	15
UML History.....	16
Case Study.....	32

DATA FLOW DIAGRAM

With **Visual Case**, you can create **data flow diagrams** both complex and simple in very little time, in an intuitive and friendly diagramming environment.

Data flow diagrams are useful in modeling high-level systems and processes. These diagrams show the way in which data flows between elements in a system and between the system and external entities.

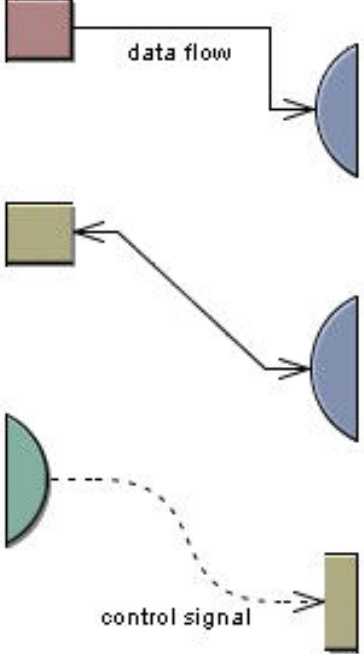
Typically, several data flow diagrams are used to understand a problem. The top-level diagram is called a context diagram and contains one Functional Transform which represents the entire system. The context diagram also shows the external entities that interact with the system. The following context diagram shows an order entry system that interacts with customers and shippers. The solid lines represent the flow of data.



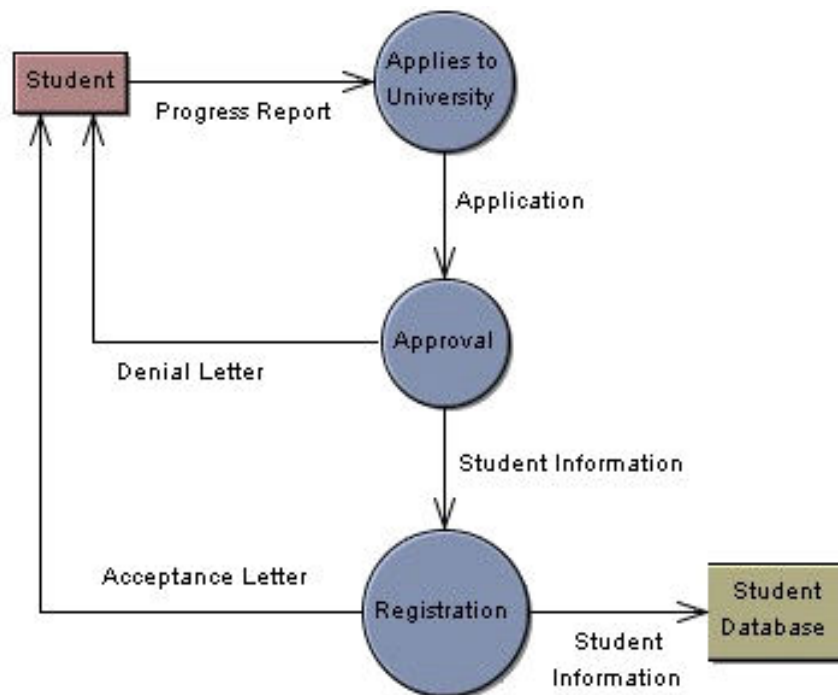
Data flow diagrams in Visual Case can model the **flow of control** as well as the flow of data. You can easily show and hide the control transforms and control signals on your diagrams at any time. A Control Transform can be decomposed into a State Diagram. Together, the state diagram and control transform model the dynamic aspects of the system.

Visual Case provides you with the following objects with which to build data flow diagrams:

	The external entities are the sources and destinations of a system's input and output. They are objects that are not within the system, but with which the system interacts.
	Data storages represent entities in the system in which data is stored. Examples include databases and files.
	A functional transform represents an action taken on the data. This may include a calculation, conversion, or other task performed.
	A control transform can send and receive control signals that help to explain the control over the data flowing through the system. In Visual Case you can optionally show and hide this flow of control at any time.

	Notes are used to add a description on the diagram for an object. In Visual Case, you can connect a note to any object using a note link.
	Data flows represent the movement of information through the system. Control signals connect control transforms to other objects on the diagram and represent control over the data. In Visual Case, you can label data flows and control signals with any text that you like. You can also choose between three styles of link: segmented, direct and quadratic (shown from top to bottom respectively). You can also select to show arrows at either, both or neither ends of a link.

Example of a Data Flow Diagram



Above is an example of a data flow diagram created with Visual Case. Some important things to note are:

- The two links labelled "Student Information" have been mapped to the same data flow. If you change the name of one, the other will automatically change. Most objects in Visual Case can easily be mapped to other entities
- The processes are all high level tasks that could be exploded into sub-data flow diagrams that clarify and further specify the task
- The data flows are all labeled with the information that is being transferred

REQUIREMENTS FROM DIAGRAMS

Functional requirements can be depicted by various types of diagrams. Each element of the diagram should be associated with the requirement it depicts. If there is a gap, either the requirements or the diagram needs correction.

Text requirements can be extracted from diagrams. The diagram structure can also be used to organize text requirements. Be sure to create traceability from diagrams to text requirements. The example below illustrates the process.

Extracting Requirements from DFDs

Data flow diagrams depict the flow of data and its transformation. Through decomposition, greater detail is revealed and documented in layers of DFDs. A numbering system is used to hierarchically relate the process layers. Data stores are also numbered so they can be traced to data models. There are four diagram components: processes, data stores, external entities, and data flows.

Processes have at least one input and one output. The process is usually symbolized by a "bubble" or similar figure. The process is what transforms the data. Functional primitives are at the lowest level of the DFDs and can be decomposed no further. You know you have gone past the functional primitive when the result sounds like program logic (if, then; case, case). Data stores are used to represent data structures or logical data files. They are often represented as open-ended shapes.

External entities represent interfaces external to the system. These are often represented as a small shape, such as a diamond.

Data flows represent the exchange of data between processes, processes and data stores, and processes and external entities. The direction of the data flow defines how data flows through the system. Data flow direction is represented by arrows.

To extract requirements from the data flow diagrams, create a template. This will make the work go quickly and ensure consistency. Insert the exact names from the diagram into the template. Users will be interested mainly in the processes and external inputs and outputs. Non-functional requirements can't be extracted from DFDs. Sample template:

Inputs from external entity: The system shall receive (DFD data name) from (DFD entity name).

Outputs to external entity: The system shall produce (DFD output name) to (DFD entity name).

Processes: The system shall (DFD process name).

Internal inputs and outputs can also be specified:

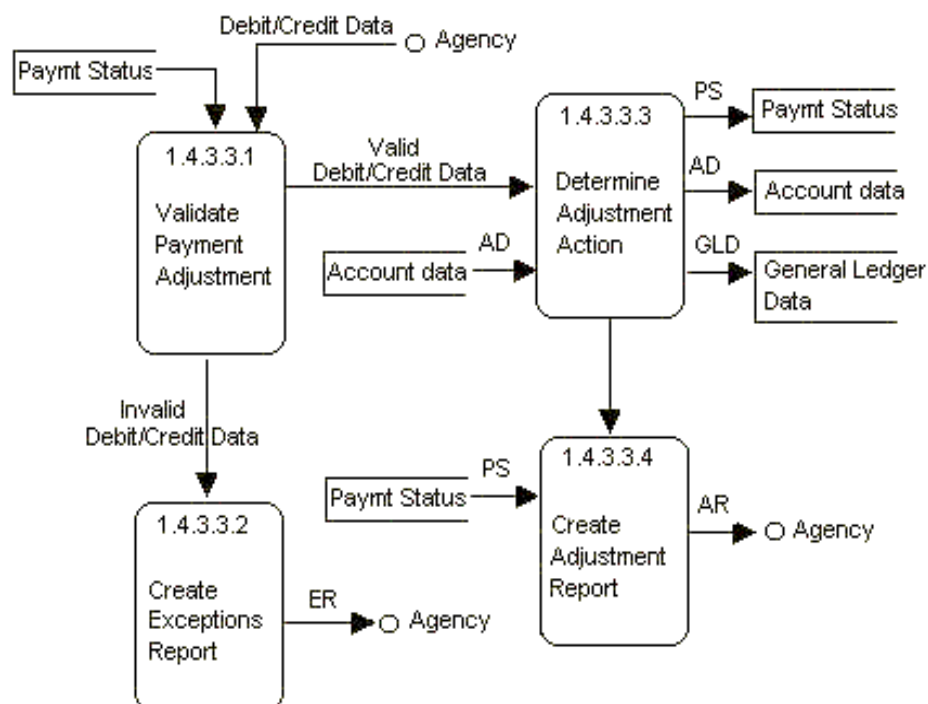
Outputs to internal entity: The system shall produce (DFD data name) to (DFD entity name).

Data from store: The system shall retrieve (DFD data name) from (DFD store name).

Data to stores: The system shall store (DFD data name) in (DFD store name).

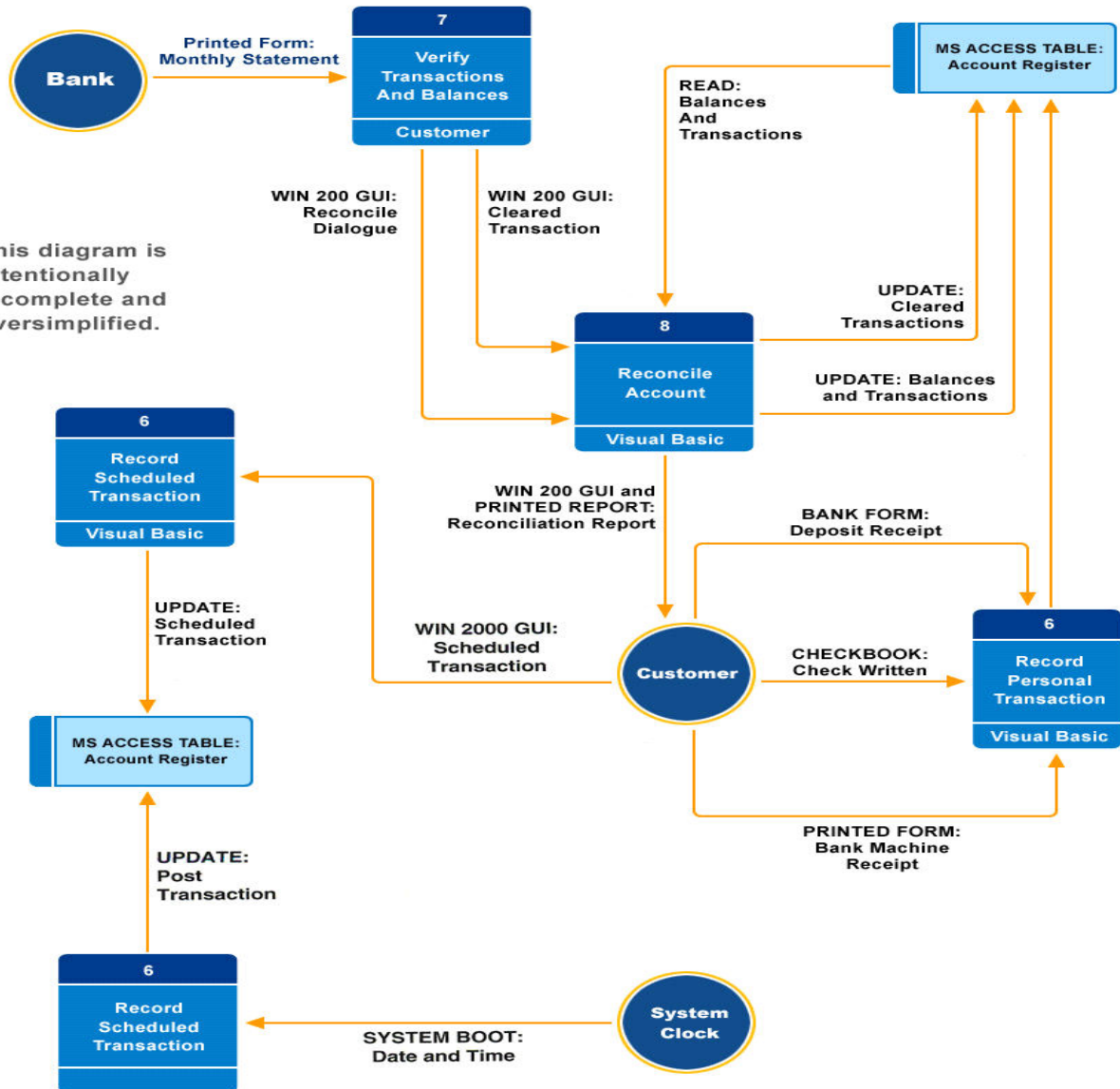
Review and edit the requirements if they are unclear. DFDs were not worded to be extracted into text. Use the DFD hierarchy to organize the requirements: higher-level processes are the parents of the primitives. Establish traceability using the DFD numbering scheme:

A simplified DFD and some requirements extracted using the template.



ID	Requirements	Model ID
F23	The system shall validate payment adjustments.	1.4.3.3.1
F24	The system shall create exception reports.	1.4.3.3.2
F25	The system shall determine adjustment actions.	1.4.3.3.3
F26	The system shall create adjustment reports.	1.4.3.3.4

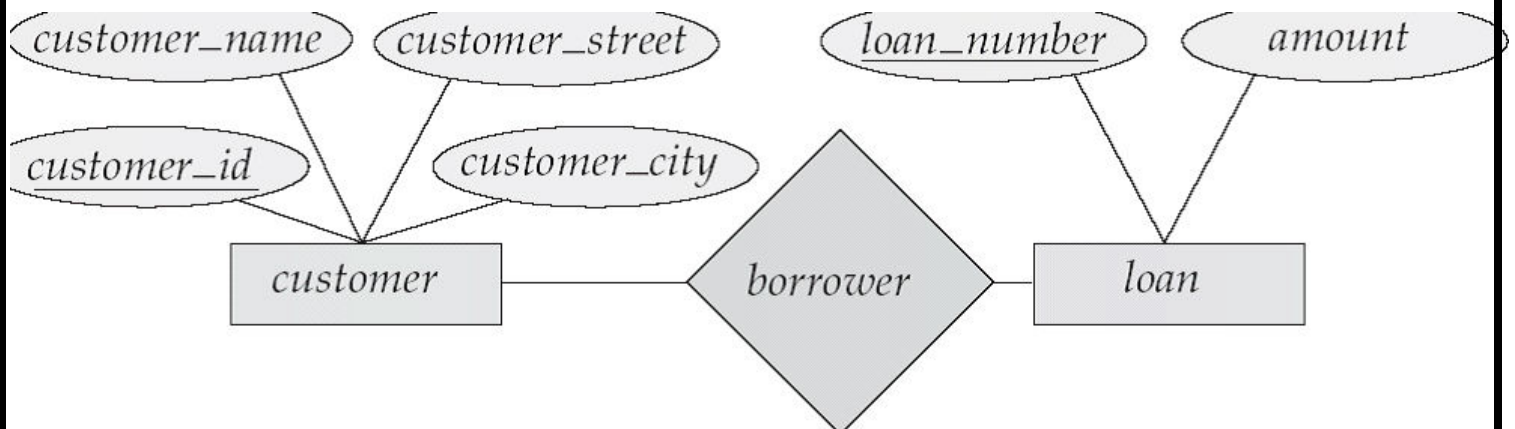
This diagram is intentionally incomplete and oversimplified.



ENTITY RELATIONSHIP DIAGRAM

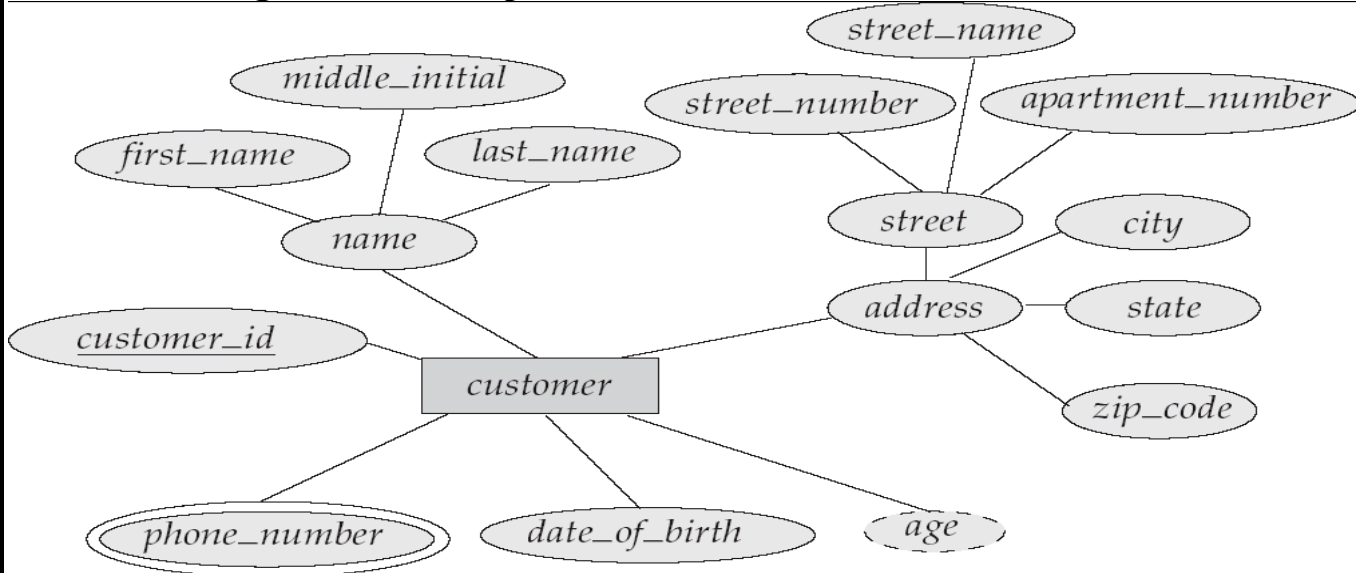
- 1) A database can be modeled as:
 - a) a collection of entities,
 - b) relationship among entities.
- 2) An **entity** is an object that exists and is distinguishable from other objects.
Example: specific person, company, event, plant
- 3) Entities have attributes
Example: people have names and addresses
- 4) An **entity set** is a set of entities of the same type that share the same properties.
Example: set of all persons, companies, trees, holidays

E-R Diagrams



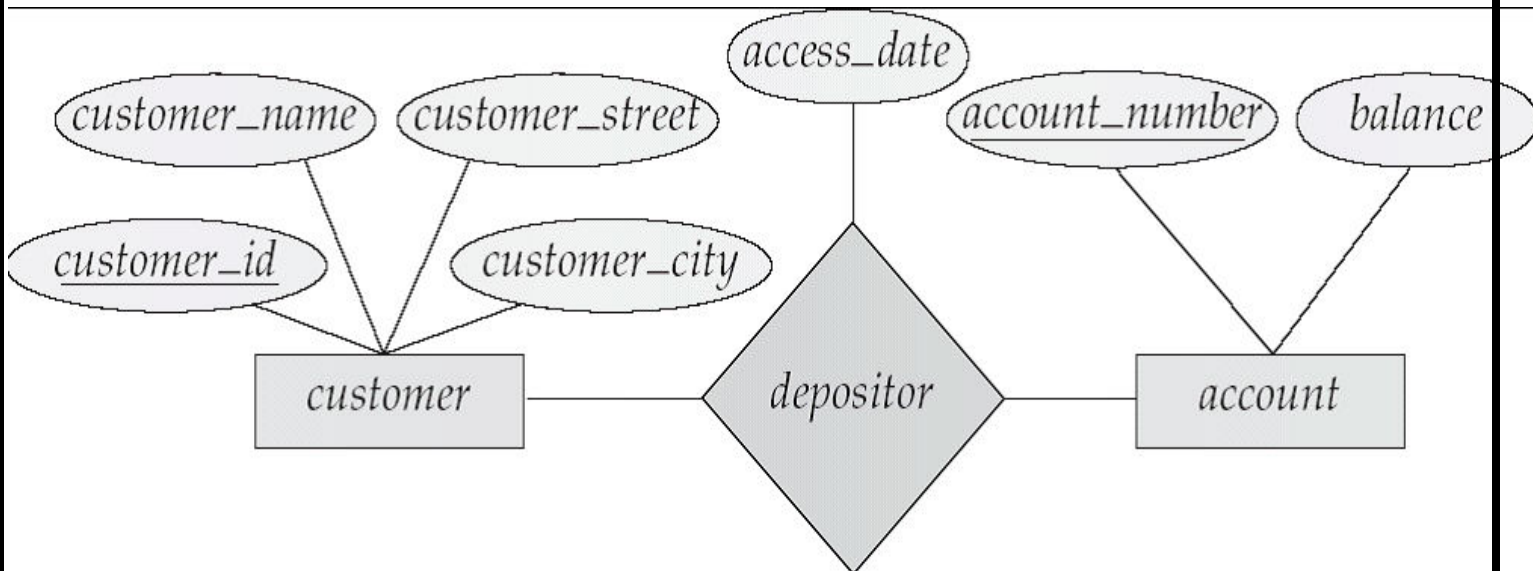
- ☐ Rectangles represent entity sets.
- ☐ Diamonds represent relationship sets.
- ☐ Lines link attributes to entity sets and entity sets to relationship sets.
- ☐ Underline indicates primary key attributes

E-R Diagram with Composite, Multivalued, and Derived Attributes



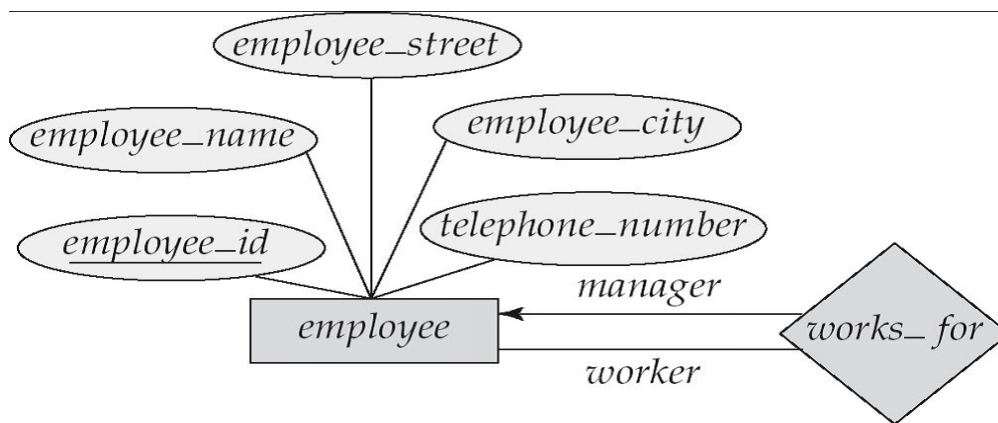
- Ellipses represent attributes
- Double ellipses represent multivalued attributes.
- Dashed ellipses denote derived attributes.

Relationship Sets with Attributes



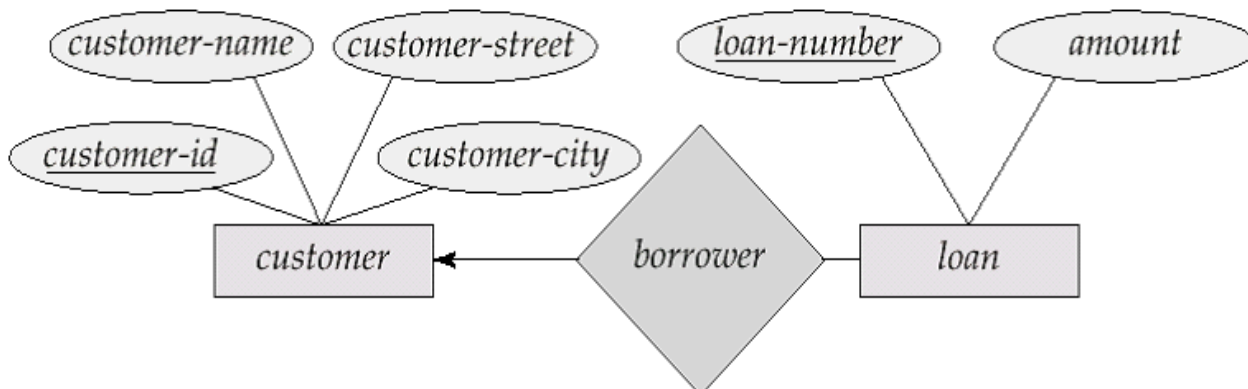
Roles

- Entity sets of a relationship need not be distinct
- The labels “manager” and “worker” are called **roles**; they specify how employee entities interact via the works for relationship set.
- Roles are indicated in E-R diagrams by labeling the lines that connect diamonds to rectangles.
- Role labels are optional, and are used to clarify semantics of the relationship



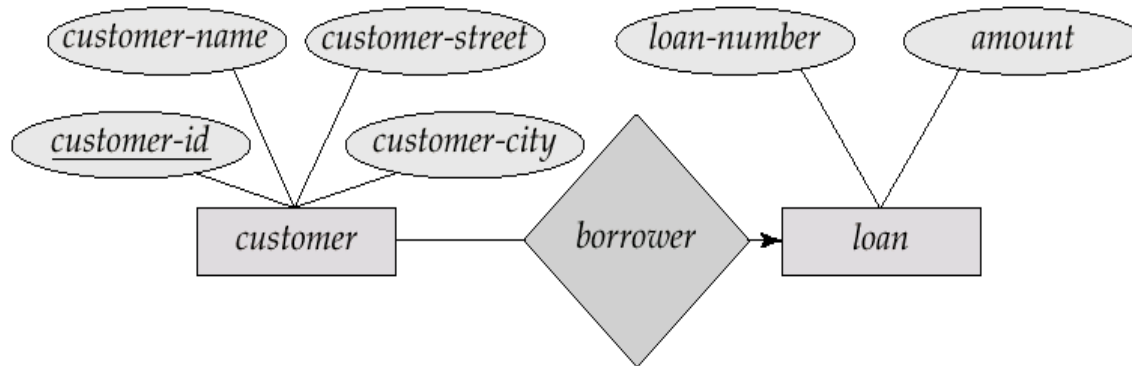
One-To-Many Relationship

- In the one-to-many relationship a loan is associated with at most one customer via borrower, a customer is associated with several (including 0) loans via borrower



Many-To-One Relationships

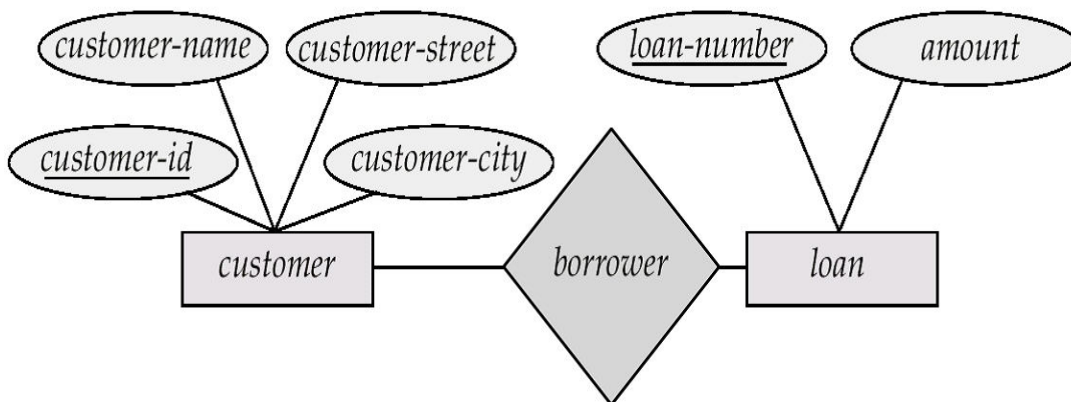
In a many-to-one relationship a loan is associated with several (including 0) customers via borrower; a customer is associated with at most one loan via borrower



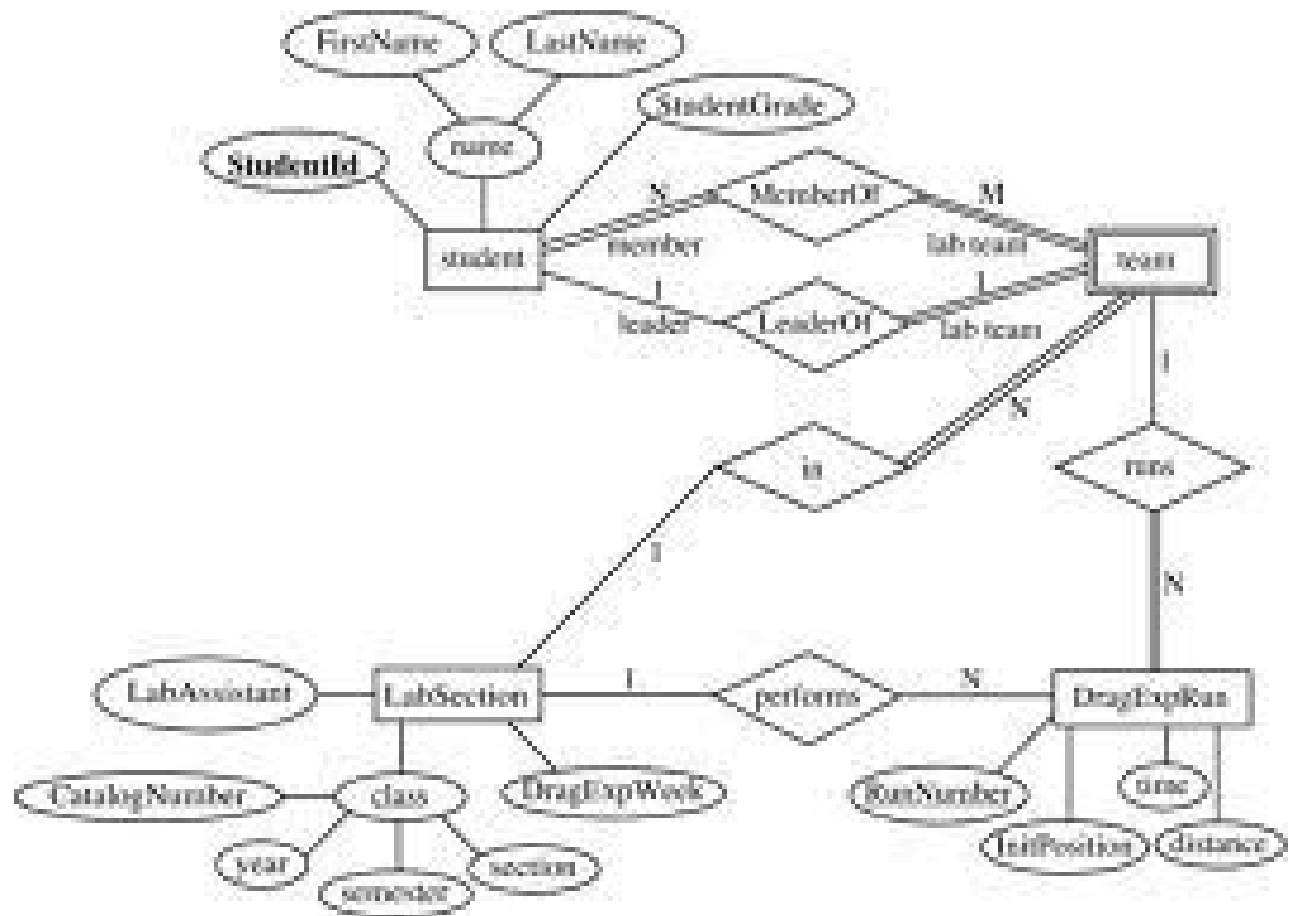
Many-To-Many Relationship

A customer is associated with several (possibly 0) loans via borrower

☐ A loan is associated with several (possibly 0) customers via borrower

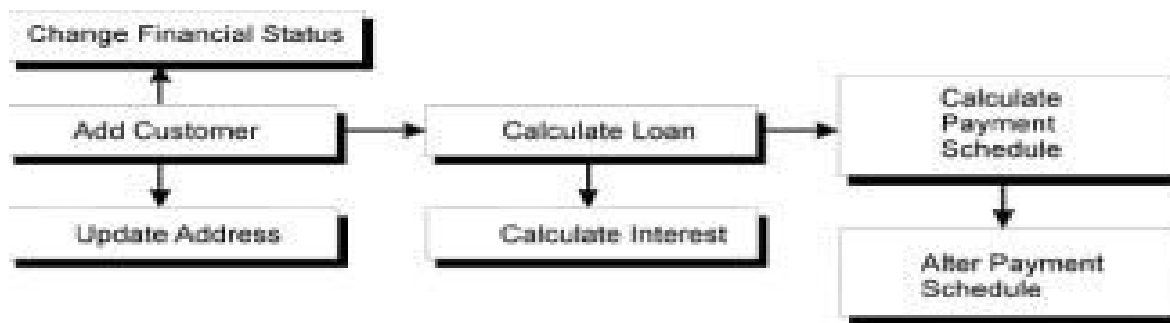


E-R Diagram Example:



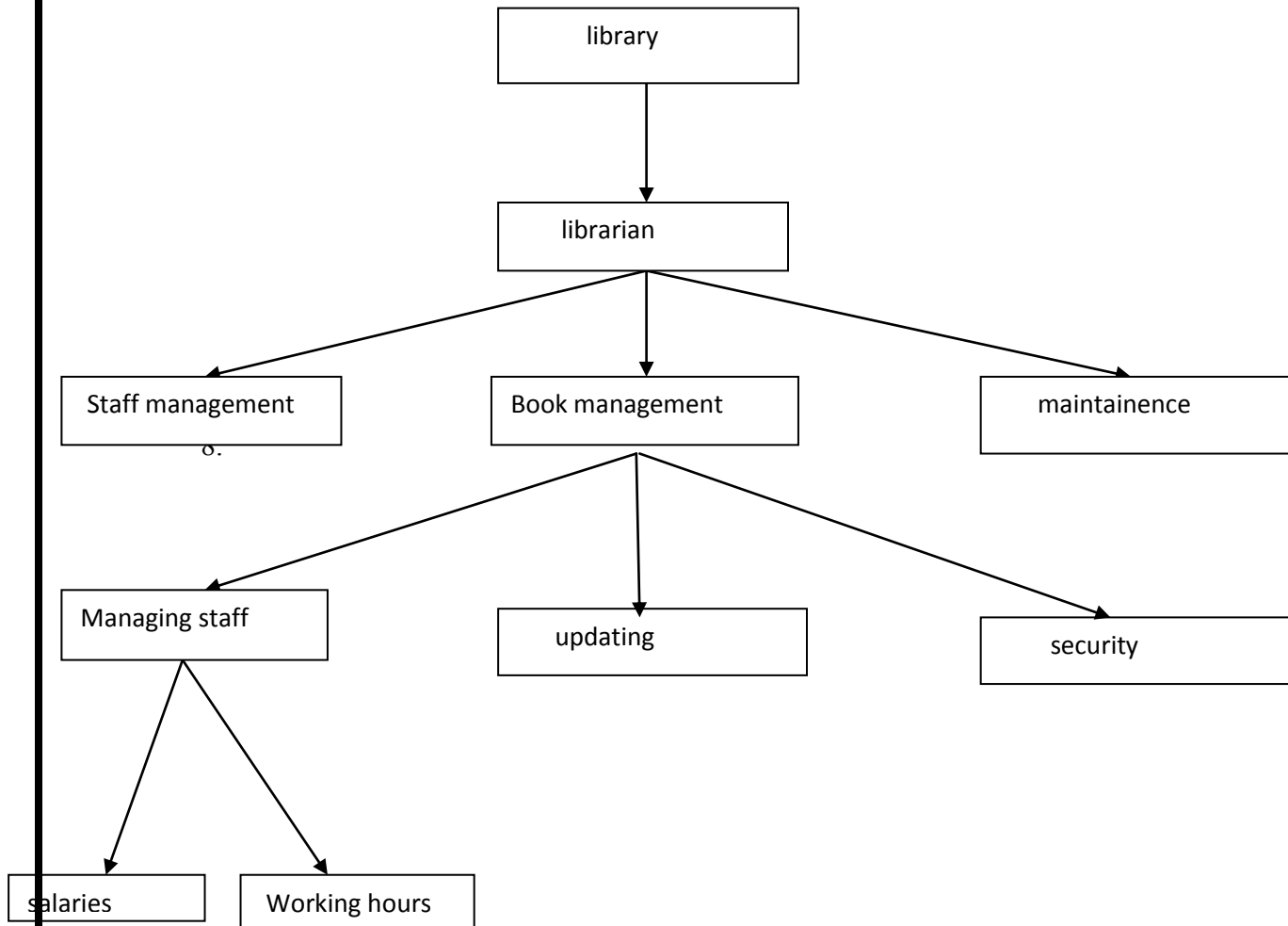
FUNCTIONAL DECOMPOSITION

Functional Decomposition is based upon discrete functions for the purpose of data manipulation, for example, the use of Jackson Structured Programming. Functions are often methods within an object-oriented environment. The partitioning of functions is based upon a different paradigm, which leads to a different grouping of functions and associated data. The simple example in figure below shows a functional version of a customer loan scenario:

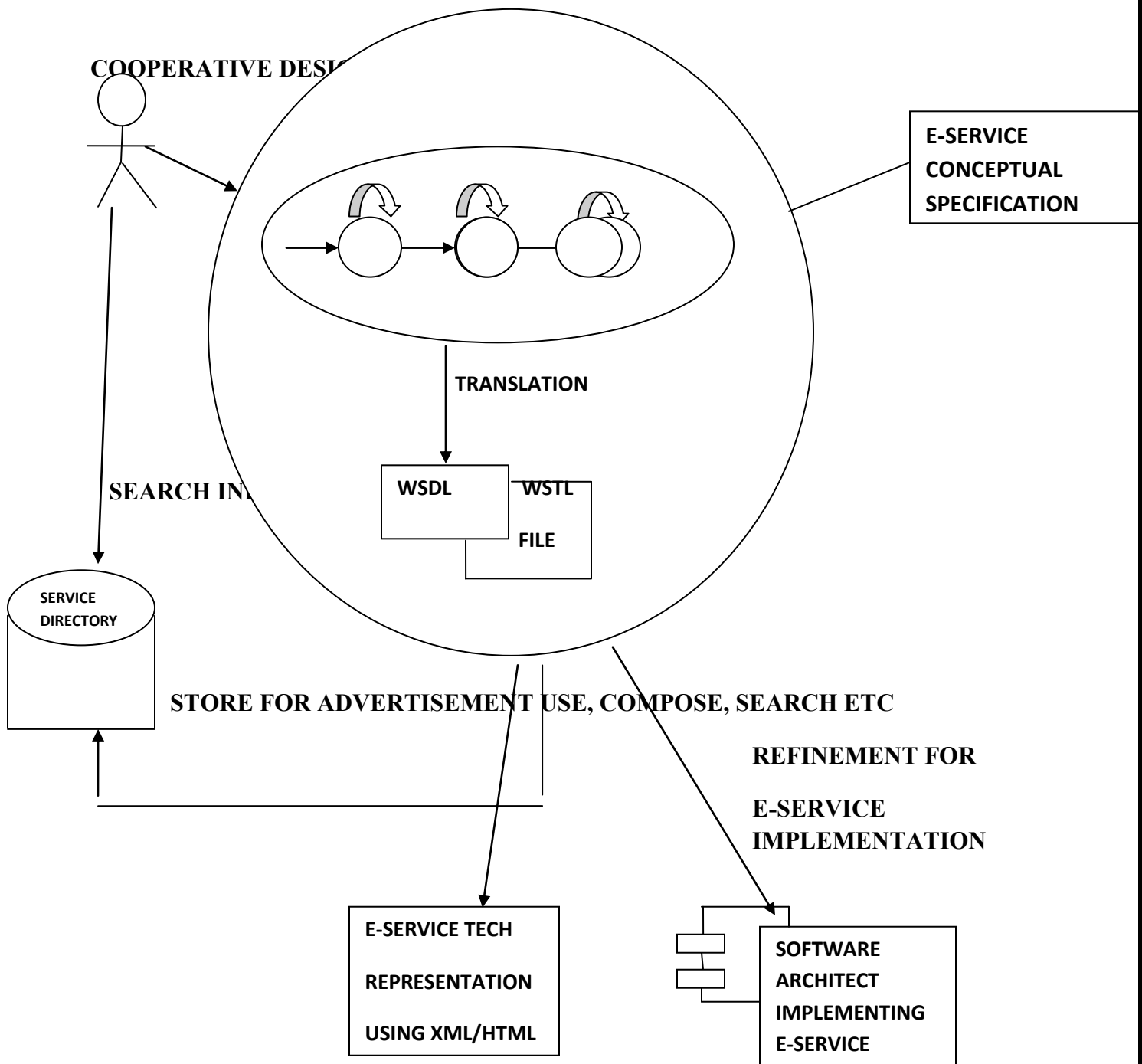


1. Adding a new customer.
2. Updating a customer address.
3. Calculating a loan to a customer.
4. Calculating the interest on a loan.
5. Calculating a payment schedule for a customer loan.
6. Altering a payment schedule.

Functional Decomposition For Library Management



DYNAMIC MODEL FOR E-SERVICE



VISUAL MODELING IN UML THROUGH RATIONAL ROSE

1. VISUAL MODELING:

Visual modeling is a way of thinking about problems using models organized around real world ideas. Models are useful for understanding problems, communication with everyone involved with the project, modeling enterprises, preparing documentation and designing programs and database. Modeling promotes better understanding of requirements, cleaner designs and more maintainable systems.

Models are abstractions that portraits the essentials of complex problem or structure by filtering out nonessential details, thus making the problem easier to understand.

2. HISTORY OF THE UML:

By unifying the notations used by these objects oriented methods, the unified modeling language provides a standard in the domain of object oriented analysis and design founded on a wide base of user experience.

The UML is an attempt to standardize the artifacts of analysis, design, semantic models, syntactic notation and diagrams.

3. THE RATIONAL ROSE TOOL:

Any software development method is best supported by a tool; the rational rose product is designed to provide the software developer with a complete set of visual modeling tools for development of robust, efficient solutions to real business needs in the client/server, distributed enterprise and real time system environments.

The tool provides the capability to:

- Identify and design business objects, and then map them to software components.
- Partition services across a three-tiered service model.
- Design how components will be distributed across a network.
- Generate code frame works directly from your model.
- Use reverse engineering to create models from existing components and applications.
- Use round-trip engineering facilities to keep the designs synchronized with the value.

ACTOR:

Actor represents anyone or anything that must interact with system. An actor may represent:

- Only input information to the system
- Only receive information from the system
- Input and receive information to and from the system.

These actors are derived from the problem statement, from SRS through the customers and domain experts.

Graphical Depiction:



An actor is a stereotype of a class and is depicted as a "stickman" on a use-case diagram.

Naming:

The name of the actor is displayed below the icon.

Additional information about the actor can be viewed in the Use-Case Specification which is identical to a Class Specification with the addition of the Stereotype field set to actor.

Relationships:

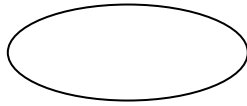
You can add relationships to the diagram to show interactions between actors and use cases. Association relationships can be drawn from an actor to a use case. The Generalize relationship can be drawn between actors.

USE CASE:

Use cases model a dialogue between an actor and the system. They represent the functionality provided by the system i.e. what capabilities will be provided to an actor by the system. The collection of use cases for a system constitutes all the defined ways the system may be used. The formal definition is: A use case is a sequence of transactions performed by a system that yields a measurable result of values of a particular actor.

Graphical Depiction:

The basic shape of a use case is an ellipse:



Use case

Naming:

A use case may have a name, although it is typically not a simple name. It is often written as an informal text description of the actors and the sequences of events between objects. Use case names often start with a verb. For example, names of possible use cases for an ATM machine might include Dispense Cash or Transfer Funds.

The name of the use case is displayed below the icon.

Relationships:

You can draw an Association relationship from a use case to an actor. You can draw a Generalize relationship between two use cases.

USE CASE REQUIREMENTS:

An association relationship may exist between an actor and a use case. This type of association is often referred to as a communicate association since it represents communication between an actor and a use case. An association may be navigable in only one direction. The navigation direction of an association represents who is initiating the communication. An association is represented as a line connecting the related elements.

Navigation is only one direction is depicted by adding an arrowhead to the association line that denotes the direction. There are two types of relationships that may exist between use cases.

- a) Include: Include relationships are created between the new use case and any other use case that “uses” its functionality.
- b) Extend: An extend relationships is used to show the following:
 - Optional behavior
 - Behavior that is run only under certain conditions such as triggering an alarm.
 - Several different flows that may be run based on actor selection.

USE CASE DIAGRAM:

A use case diagram is a graphical view of some or all of the actors, use cases and their interactions identified for a system. Each system typically has a main use case diagram, which is a picture of the system boundary (actors) and the major functionality provide by the system (use cases).

ACTIVITY DIAGRAM:

Activity diagrams are flowcharts that are used to show the work flow of a system, that is, they show the flow of control from activity to activity in the system, what activities done in parallel and any alternate paths through the flow.

Activity diagrams contain activities, transactions between the activities, division paths and synchronization bars. In the UML, activities are represented as rectangles with round edges, transactions are drawn as directed arrows, decision points are shown as diamonds and synchronization bars are drawn as thick horizontal or vertical bars.

Basic Activity Diagram Symbols and Notations:

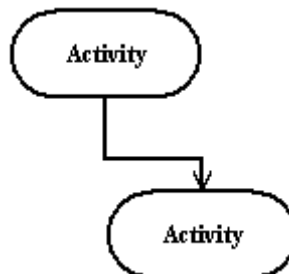
- **Action states**

Action states represent the noninterruptible actions of objects. You can draw an action state in Smart Draw using a rectangle with rounded corners.



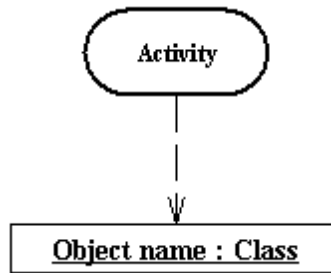
- **Action Flow**

Action flow arrows illustrate the relationships among action states.



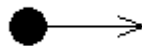
- **Object Flow**

Object flow refers to the creation and modification of objects by activities. An object flow arrow from an action to an object means that the action creates or influences the object. An object flow arrow from an object to an action indicates that the action state uses the object.



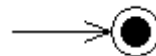
- **Initial State**

A filled circle followed by an arrow represents the initial action state.



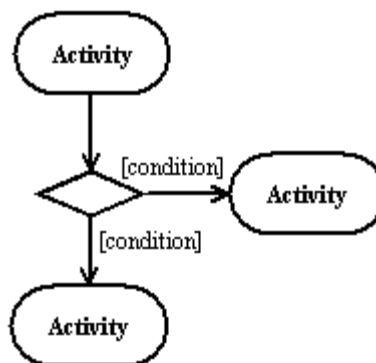
- **Final State**

An arrow pointing to a filled circle nested inside another circle represents the final action state.



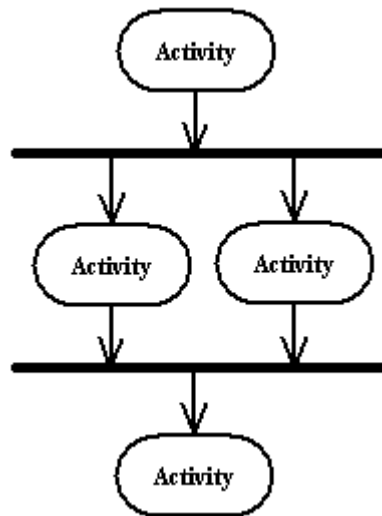
- **Branching**

A diamond represents a decision with alternate paths. The outgoing alternates should be labeled with a condition or guard expression. You can also label one of the paths "else."



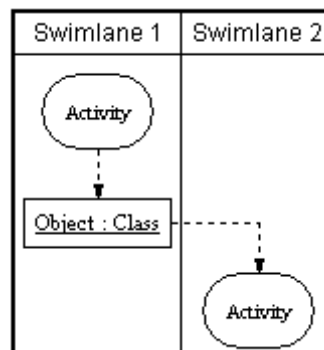
- **Synchronization**

A synchronization bar helps illustrate parallel transitions. Synchronization is also called forking and joining.



- **Swim lanes**

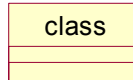
Swim lanes group related activities into one column.



PACKAGES AND CLASS DIAGRAMS:

Class

A class is description of a group of objects with common properties, common behavior, common relationships to other objects and common semantics. Thus a class is a template to create objects. Each object is an instance of some class and objects cannot be instances of more than one class.

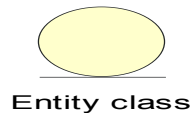


Stereotypes of Class

Classes have stereotypes. Some common stereotypes for class are entity, boundary, control utility and exception.

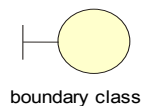
Entity Class

An entity class models information and association behavior that is generally long lived. This type of class may reflect a real world entity or it may reflect a real world entity or it may be needed to perform tasks internal to the system. Entity classes typically are classes that are needed by the system to accomplish some responsibility.



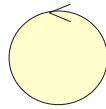
Boundary Class

Boundary classes handle the communication between the system surroundings and the inside of the system. They can provide the interface to the user or another system. They constitute the surroundings dependant part of the system. Boundary classes are used to model the system.



Control Class

Control classes model sequencing behavior specific to one or more use cases. Control classes coordinate the events needed to realize the behavior specified in the use case.

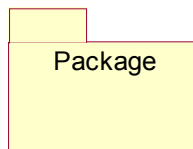


control class

Package:

A package in the logical view of the model is a collection of related packages and/or classes. By grouping classes into packages, we can look at the higher-level view of the model or we can dig deeper into the model by looking at what is contained by the package.

Each package contains an interface that is realized by its set of public classes those classes to which classes in other packages talk. The rest of classes in a package are implementation classes do not communicate with classes in the other packages.

**Class Diagram:**

Class diagrams are created to provide a picture or view of some of the classes in the model. The main class diagram in the logical view of the model is typically a picture of the packages in the system. Each package also has its own main class diagram.

SEQUENCE DIAGRAM:

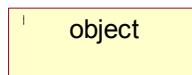
A sequence diagram shows objects interactions arrange in time sequence. It depicts the objects and classes involved in the scenario and the objects needed to carry out the functionality of the scenario. Sequence diagrams are typically associated with use cases realizations in the logical view of the system under development.

In the UML, an object in a sequence diagram is drawn as a rectangle containing the name of object underlined. An object can be named in one of three ways: the object name, the object name and its class, or just the class name. Each object also has its timeline represented by a dashed line below object. Messages between objects are represented by arrows that point from the client to the supplier.

Element and its description:

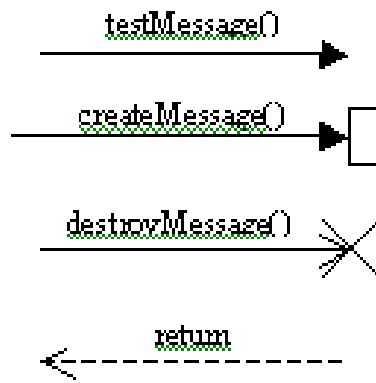
Object:

The primary element involved in a sequence diagram is an Object—an instance of a class. A Sequence diagram consists of sequences of interaction among different objects over a period of time. An object is represented by a named rectangle. The name to the left of the ":" is the object name and to its right is the class name.



Message:

The interaction between different objects in a sequence diagram is represented as messages. A message is denoted by a directed arrow. Depending on the type of message, the notation differs. In a Sequence diagram, you can represent simple messages, special messages to create or destroy objects, and message responses.



COLLABORATION DIAGRAM:

A collaboration diagram is an alternate way to show a scenario. This type of diagram shows objects interaction organized around the objects and their links to each other. A collaboration diagram contains:

- a) Objects drawn as rectangles.
- b) Links between objects shown as lines connecting the linked objects.
- c) Messages shown as text and an arrow that points from client to the supplier.

Basic Collaboration Diagram Symbols and Notations:

Class roles

Class roles describe how objects behave. Use the UML object symbol to illustrate class roles, but don't list object attributes.

Object : Class

Association roles

Association roles describe how an association will behave given a particular situation. You can draw association roles using simple lines labeled with stereotypes.

<<global>>

Messages

Unlike sequence diagrams, collaboration diagrams do not have an explicit way to denote time and instead number messages in order of execution. Sequence numbering can become nested using the Dewey decimal system. For example, nested messages under the first message are labeled 1.1, 1.2, 1.3, and so on. The a condition for a message is usually placed in square brackets immediately following the sequence number. Use a * after the sequence number to indicate a loop.

1.4 [condition]:
message name



1.4 * [loop expression] :
message name

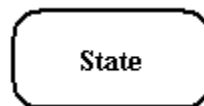


STATE CHART DIAGRAM:

A state chart diagram shows the status of the single object, the events, the messages that cause a transaction from one state to another and the actions that result from a state change. A state chart diagram encompasses all the messages that an object can send and receive. Scenarios represent the path following through the state chart diagram.

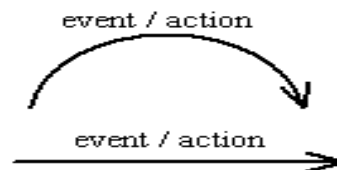
States

States represent situations during the life of an object. You can easily illustrate a state in SmartDraw by using a rectangle with rounded corners.



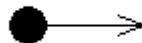
Transition

A solid arrow represents the path between different states of an object. Label the transition with the event that triggered it and the action that results from it.



Initial State

A filled circle followed by an arrow represents the object's initial state.



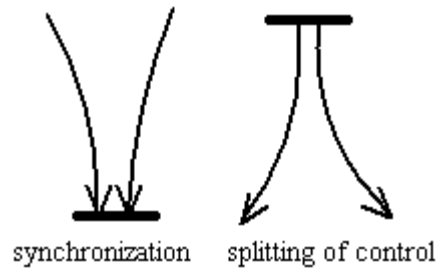
Final State

An arrow pointing to a filled circle nested inside another circle represents the object's final state.



Synchronization and Splitting of Control

A short heavy bar with two transitions entering it represents a synchronization of control.
A short heavy bar with two transitions leaving it represents a splitting of control that creates multiple states.



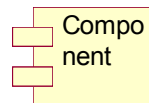
COMPONENT DIAGRAM:

The main component diagram is typically a view of the packages identified in the logical view. The classes identified in the logical view, which were grouped under the packages, are mapped as the components in the component view. Each component thus created as per the mapping of the respective class has to be assigned the source code option offered by the Rational Rose Tool. (Ex: ANSI, C++, VB, Java, VC++ is identified as per the implementation required).

Basic Component Diagram Symbols and Notations:

Component

A component is a physical building block of the system. It is represented as a rectangle with tabs.



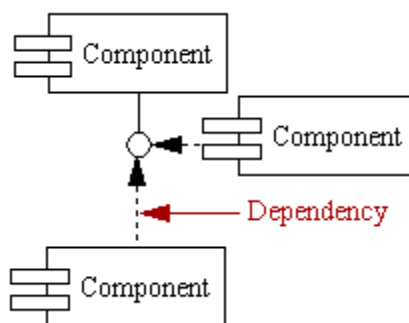
Interface

An interface describes a group of operations used or created by components.



Dependencies

Draw dependencies among components using dashed arrows.



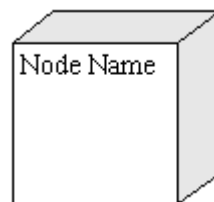
DEPLOYMENT DIAGRAM:

This view of architecture involves mapping software to processing nodes. Deployment diagrams are created to show the different nodes along with their conditions in the system. The deployment diagram visualizes the distribution of components across the enterprise. The diagram allows the architecture team to understand the overall system topology and helps in mapping the internal components to the executable process.

Basic Deployment Diagram Symbols and Notations:

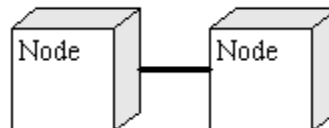
Node

A node is a physical resource that executes code components.



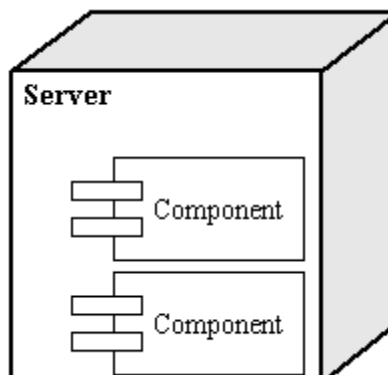
Association

Association refers to a physical connection between nodes, such as Ethernet.



Components and Nodes

Place components inside the node that deploys them.



CASE STUDY

Software Requirements Specification For E-mail Service

Table of Contents

Table of Contents	33
1. Introduction.....	34
1.1 Purpose.....	34
1.2 Document Conventions.....	34
1.3 Intended Audience	34
1.4 E-mail Service Scope.....	34
2. Overall Description.....	35
2.1 Product Perspective.....	35
2.2 Product Features.....	35
2.3 User Classes.....	36
2.4 Operating Environment.....	36
2.5 Design and Implementation Constraints.....	36
3. System Features	37
3.1 E-mail Client.....	37
3.2 E-mail Server's.....	Error! Bookmark not defined.
4. External Interface Requirements	39
4.1 Hardware Interfaces	39
4.2 Software Interfaces	39
4.3 Communications Interfaces	39
5. Other Nonfunctional Requirements.....	39
5.1 Security Requirements	Error! Bookmark not defined.
5.2 Software Quality Attributes	40

Introduction

Purpose

This is the Software Requirements Specification (SRS) for the E-mail Service. The purpose of this SRS is to give an understanding into the E-mail service describing the functionality of the Email Service (the most popular and easiest way of communicating in these modern days), the technology which has changed the way the people communicate. An E-mail is an electronic message transmitted over a network from one user to the other, its the best, reliable, easy, economical, fast way of communicating. This SRS covers the entire email service from i.e, an email from one client to the other.

Document Conventions

The document conventions are that the font used to write this SRS is Arial and size is 10 , and for headings the font used is Times and size is 14.

Intended Audience

The audience of this E-mail service is every one who is computer literate and knows a bit of internet browsing, and every one who wants to communicate in a better way than the usual and conventional way. This SRS contains complete information about the E-mail Service how the mails are composed and how they are transmitted etc.

E-mail Service Scope

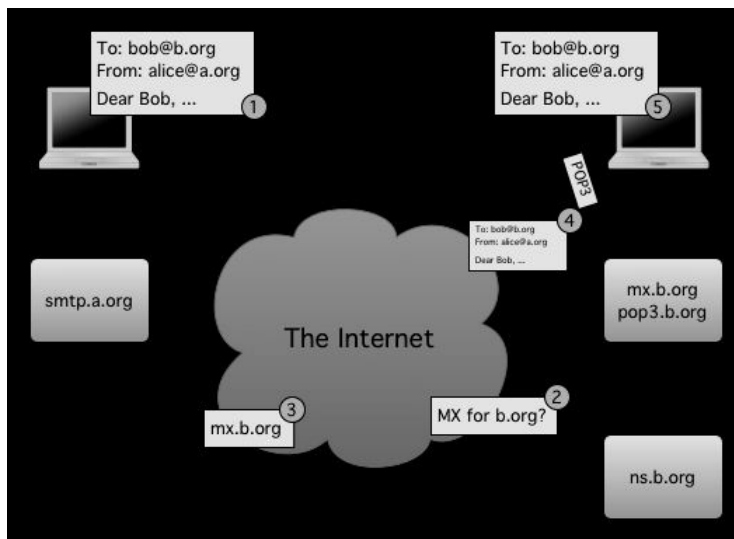
Electronic mail, abbreviated e-mail or email, is a method of composing, sending, and receiving messages over electronic communication systems. Most e-mail systems today use the Internet. Cheapest, and fastest means of communication. No other means of communication is as economical as e-mail.

- It's easy to send your message to more than one person.
- Most e-mail systems have a reply button that enables to include all or part of the original message when you are writing a reply
- One can attach drawings, sounds, video clips, and other computer files to your e-mail.
- One can easily save thousands of e-mail messages, and can search message files electronically.
- One can paste all or part of an e-mail into other computer documents.

Overall Description

Product Perspective

E-mail predates the Internet; existing e-mail systems were a crucial tool in creating the Internet. E-mail started in 1965 as a way for multiple users of a time-sharing mainframe computer to communicate. The exact history is murky. E-mail was quickly extended to become *network e-mail*, allowing users to pass messages between different computers. The ARPANET computer network made a large contribution to the evolution of e-mail. Ray Tomlinson initiated the use of the @ sign to separate the names of the user and their machine in 1971. E-mail became an increasingly important feature of work group collaboration products developed by vendors such as Wang, Lotus, IBM, and Microsoft. These systems often provided enhanced e-mail features (such as file attachments, Rich Text Format, and delivery confirmation), but only when sending e-mail to other users of the same system.



Product Features

E-mail client:

The client carries out the user's interactions with the E-mail server. Clients can appear in various forms:

Application_based – these are installed onto users' machines and include Microsoft Outlook and the freely available Outlook Express and Eudora.

Web_based – these appear in a web browser's window and include Hotmail, Yahoo and Outlook web client. Clients vary greatly in functionality, but all provide a basic level of functionality that assists the user.

E-mail Server:

An E-mail server is typically a combination of processes running on a server with a large storage capacity – a list of users and rules, and the capability to receive, send and store emails and attachments.

SMTP Server:

Simple mail transfer protocol was invented during the 1980s as a standard for the transmission of messages. It is the language that most mail servers use to send messages between each other.

POP3 Server:

POP3 (post office protocol 3) is used by clients to collect E-mail from E-mail servers. Clients must supply a username and password to the server in order to log into their account or POP3 mailbox.

IMAP Server

IMAP is the internet message access protocol used for accessing E-mail stored on a server. It is a protocol that allows users to perform certain electronic mail functions on a remote server rather than on their local computer.

User Classes

The E-mail service was being used by all user classes and will be used for many more years, the user classes include the professionals, organizations, students, and every one.

Operating Environment

The most important part of E-mail Service is the E-mail Client which runs on the user system which can be any thing running on Intel processors or Mac systems etc and the Operating system can be anything like Windows , Linux , Mac etc ..they should only have the Browser like Internet Explore, Mozilla Fire fox, Netscape Navigator, Opera etc ..for opening the web based email client and for the application based E-mail clients they are already installed in the user systems like Microsoft Outlook etc..and for the transmission of the e-mail the SMTP , POP3 , IMAP servers are used which are the hardware running on any Operating System mostly on Linux, Windows etc ...and for sending the attachments with the email the MIME (multi-purpose internet mail extensions) is used

Design and Implementation Constraints

The E-mail Clients have to be designed in such a way that they must be compatible with all the modern and popular browsers. And the servers have to be running all the time, and the protocols to be used are the TCP/IP protocol. And the E-mail service should maintain a database for storing the E-mail id's and the passwords and the user information and it should even give a storage space and a database for each user for Address book etc.

System Features

E-mail Client:

3.1.1 Description and Priority

The client carries out the user's interactions with the E-mail server. Clients can appear in various forms:

Application_based – these are installed onto users' machines and include Microsoft Outlook and the freely available Outlook Express and Eudora.

Web based – these appear in a web browser's window and include Hotmail, Yahoo and Outlook web client. Clients vary greatly in functionality, but all provide a basic level of functionality that assists the user. Functions include the ability to create new emails, display and store received emails, hold address lists of contacts, a calendar, journal and other extra functions that help organize the user's working day. The client is also configured with the account information and names or IP addresses of the E-mail servers with which it will be communicating in order to send and receive E-mails.

And the E-mail Client is of high priority in a email service the most important one.

3.1.2 Stimulus/Response Sequences

An E-mail from one client user to another: A sender called Harish Kumar B.A. Founder, The Leo Technologies . His E-mail address is harishkumarba@bheemarthi.com. He sends an E-mail to Bill Gates Founder, Microsoft Inc., Seattle. Harish starts his E-mail client and selects 'Compose new message'. Harish types Bill Gates's E-mail address 'billgates@microsoft.com' into the 'To' box. He types in his subject and clicks the 'Send' button. The client makes a connection to its local E-mail server 'mail.bheemarthi.com' using SMTP port 25. The client passes the E-mail's sender and recipient addresses along with the main body of the message to the E-mail server.

3.1.3 Functional Requirements

REQ-1: System which can be any thing running on Intel processors etc or Mac systems etc

REQ-2: Operating system can be anything like Windows, Linux , Mac etc

REQ-3: The Browser like Internet Explore, Mozilla Fire fox, Netscape Navigator, Opera etc

Email Server's:

3.2.1 Description and Priority:

An E-mail server is typically a combination of processes running on a server with a large storage capacity – a list of users and rules, and the capability to receive, send and store emails and attachments. These servers are designed to operate without constant user intervention and should happily process emails for months as sending, receiving and maintenance tasks are carried out at scheduled times.

3.2.2 Stimulus/Response Sequences

The client passes the E-mail's sender and recipient addresses along with the main body of the message to the E-mail server. When the server has received the E-mail it looks at the domain part of the recipient's address, the part after the @ symbol – 'microsoft.com'. If the domain is local to the server – 'bheemarthis.com' – it will simply pass the message onto the local POP3 server for delivery. As the recipient's domain is not local to the E-mail server, it passes it onto the E-mail server that looks after that specific domain. It does this by asking another server or service called DNS (domain names server) for the IP (internet protocol) address or MX (mail exchange) record of the E-mail server that deals with E-mail for 'microsoft.com'. The DNS replies with an IP address for the other domain's SMTP server. Once the E-mail server has this IP address, it opens up a connection to the recipient's SMTP server on port 25 and passes the message on. Once the receiving SMTP server has the E-mail, it checks the domain part of the E-mail address recognizes it as its own local domain and passes it internally to the POP3 server. The POP3 server holds the message in Bill Gates mailbox. The POP3 mailbox holds the messages in a queue until the user, in this case Bill Gates, connects to the E-mail server on port 110 to collect mail. He downloads his queued POP3 mailbox from the server to the client and receives Harish's E-mail.

- 3.2.3 Functional Requirements
- REQ-1: SMTP Server
 - REQ-2: POP3/IMAP Server

External Interface Requirements

Hardware Interfaces

The hardware interfaces used for E-mail Service are the E-mail server's like SMTP Server, POP3 Server, IMAP Server etc. And again for sending the attachments using the email the MIME (multi-purpose internet mail extensions) is used.

Software Interfaces

The software interfaces used for E-mail Service are the E-mail Client's like Application based mail clients and the Web based email clients.

Communications Interfaces

Interfaces used here for communication are the SMTP Server, POP3 Server, IMAP Server etc. And the other type of interfaces are the , web browser, network server communications protocols like TCP/IP protocol, and other type of protocol used is HTTP which is used by the browser to get the web based email client onto the user machine.

Other Nonfunctional Requirements

Security Requirements

E-mail privacy, some security precautions, cannot be compromised because

- E-mail messages are generally not encrypted.
- E-mail messages have to go through intermediate computers before reaching their destination, meaning it is relatively easy for others to intercept and read messages.
- Many Internet Service Providers (ISP) store copies of your e-mail messages on their mail servers before they are delivered. The backups of these can remain up to several months on their server, even if we delete them in our mailbox.

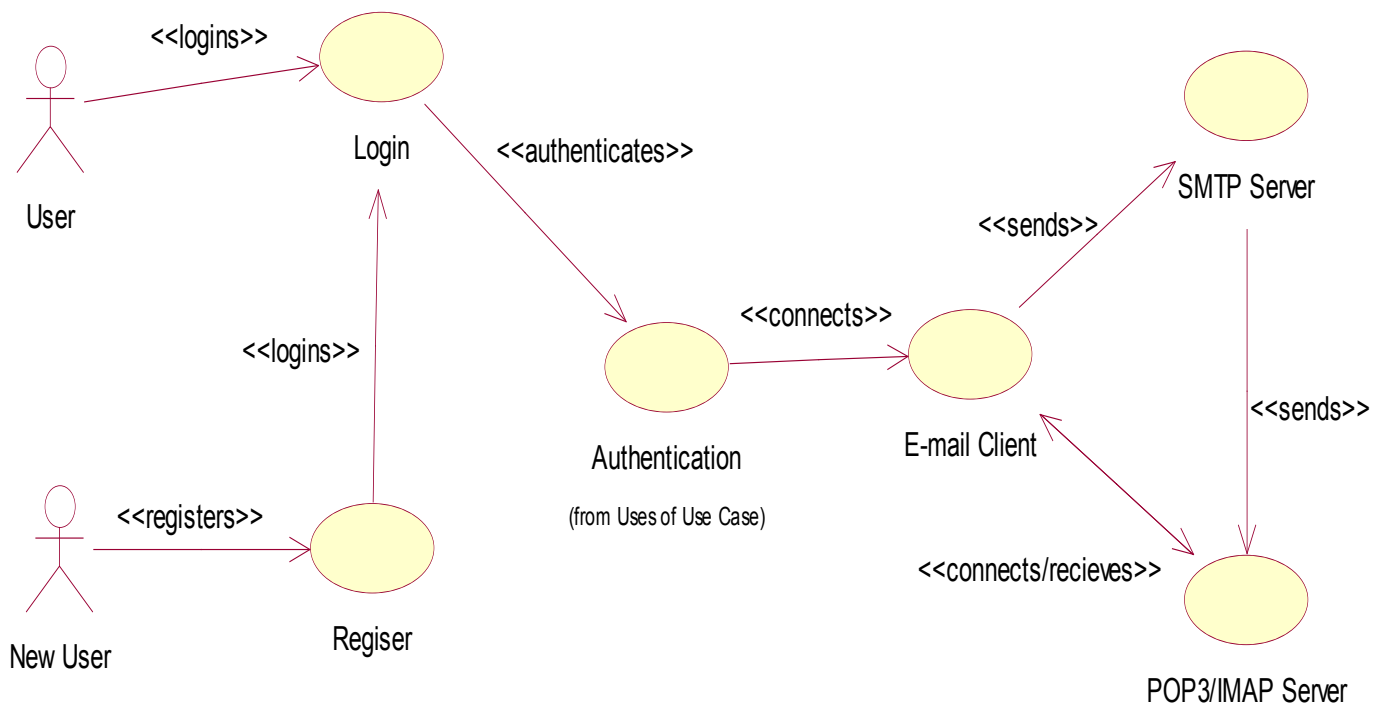
There are cryptography applications that can serve as a remedy to the above, such as Virtual Private Networks, message encryption using PGP or the GNU Privacy Guard, encrypted communications with the e-mail servers using Transport Layer Security and Secure Sockets Layer, and/or encrypted authentication schemes such as Simple Authentication and Security Layer.

Software Quality Attributes

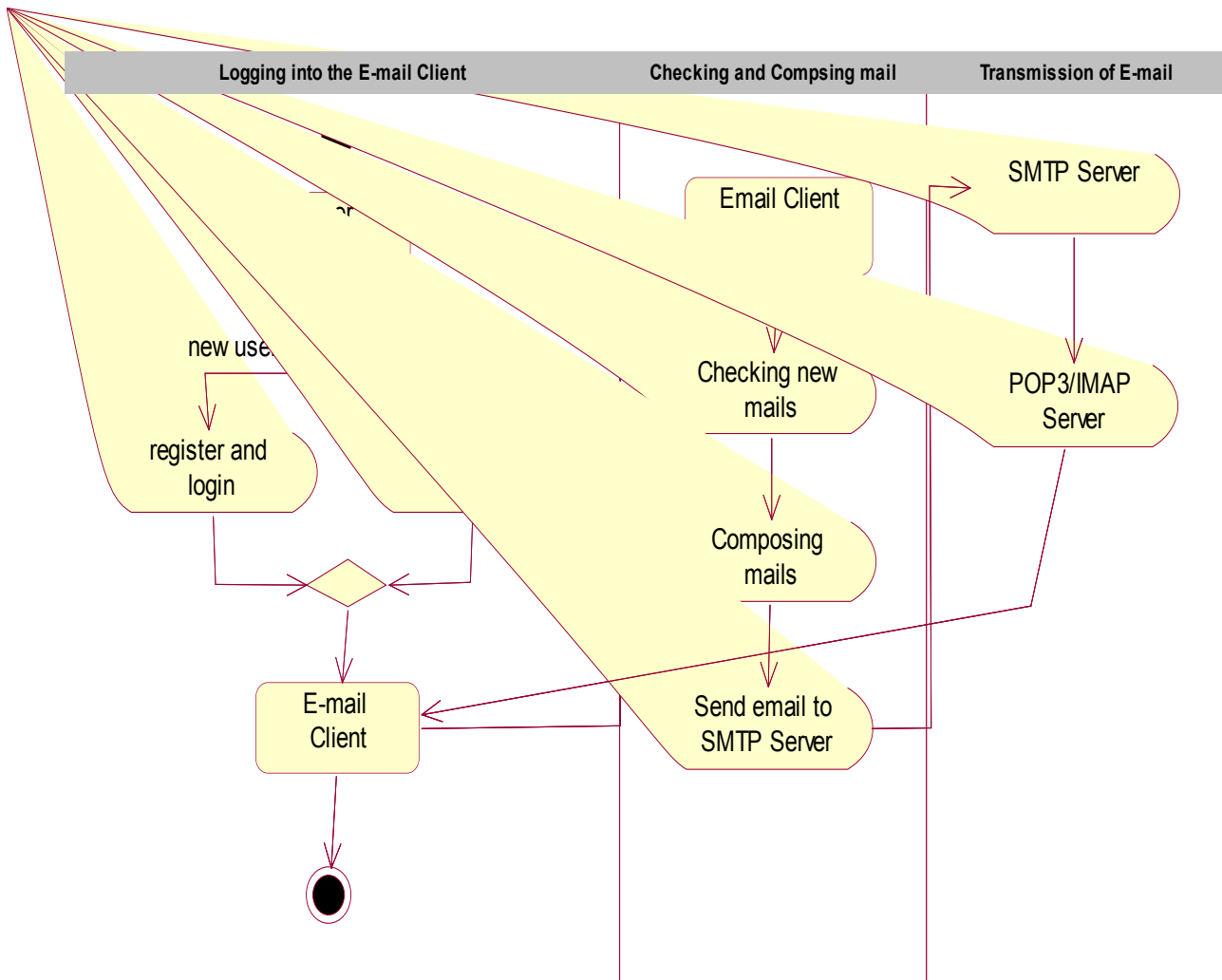
The best, reliable, easy, economical, fast way of communicating is by using the E-mail Service; no other communication service is so flexible, easy and economical. Ease of use of E-mail Service is that anyone who is computer literate and knows a bit of internet browsing can send an email in seconds.

DIAGRAMS

USECASE DIAGRAM

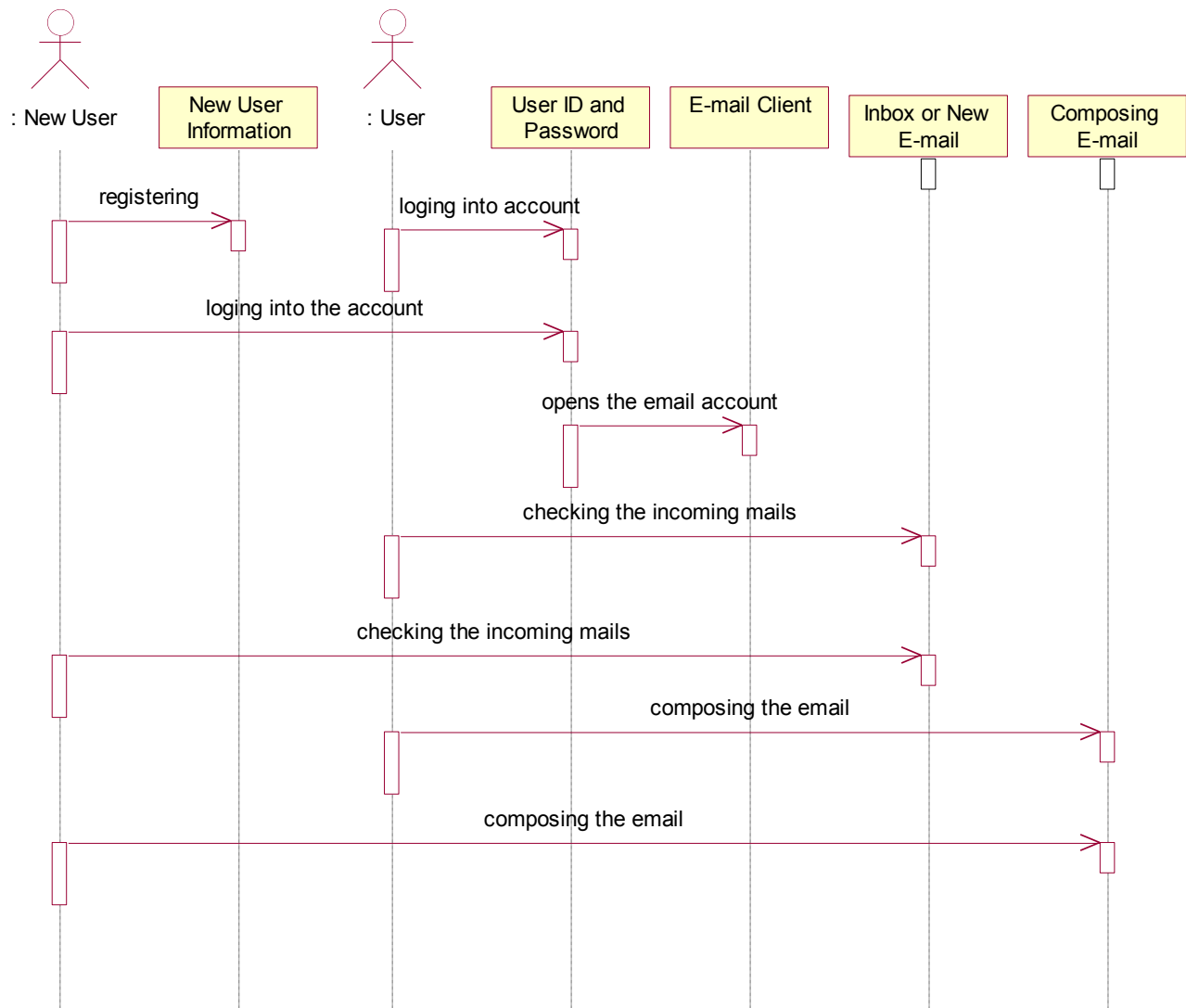


ACTIVITY DIAGRAMS



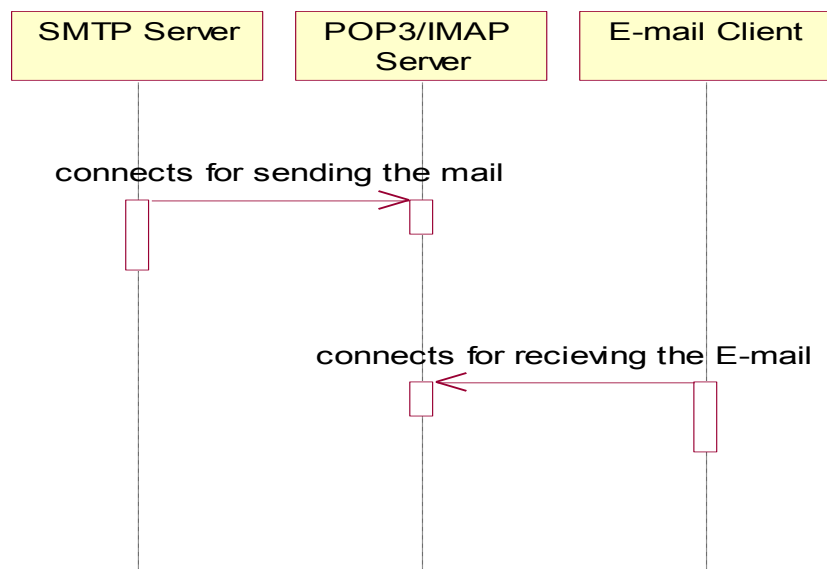
SEQUENCE DIAGRAM

E-mail Client/Logging into the E-mail Client:

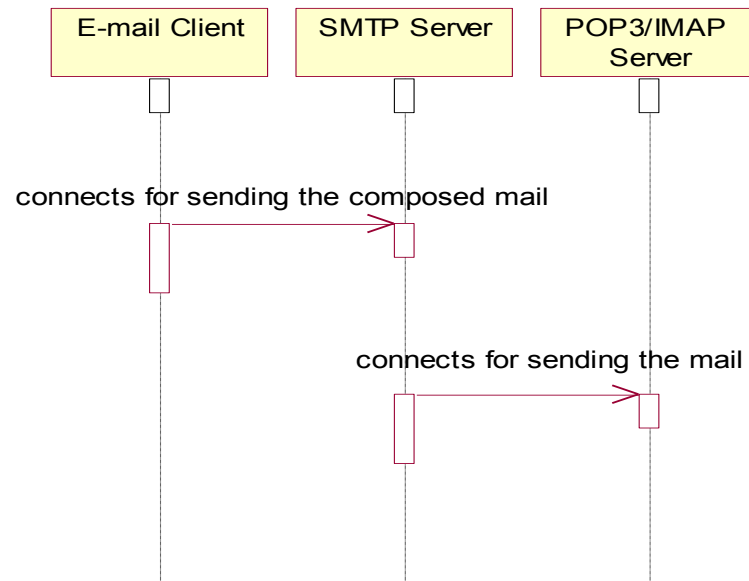


SEQUENCE DIAGRAM

POP3/IMAP Server:

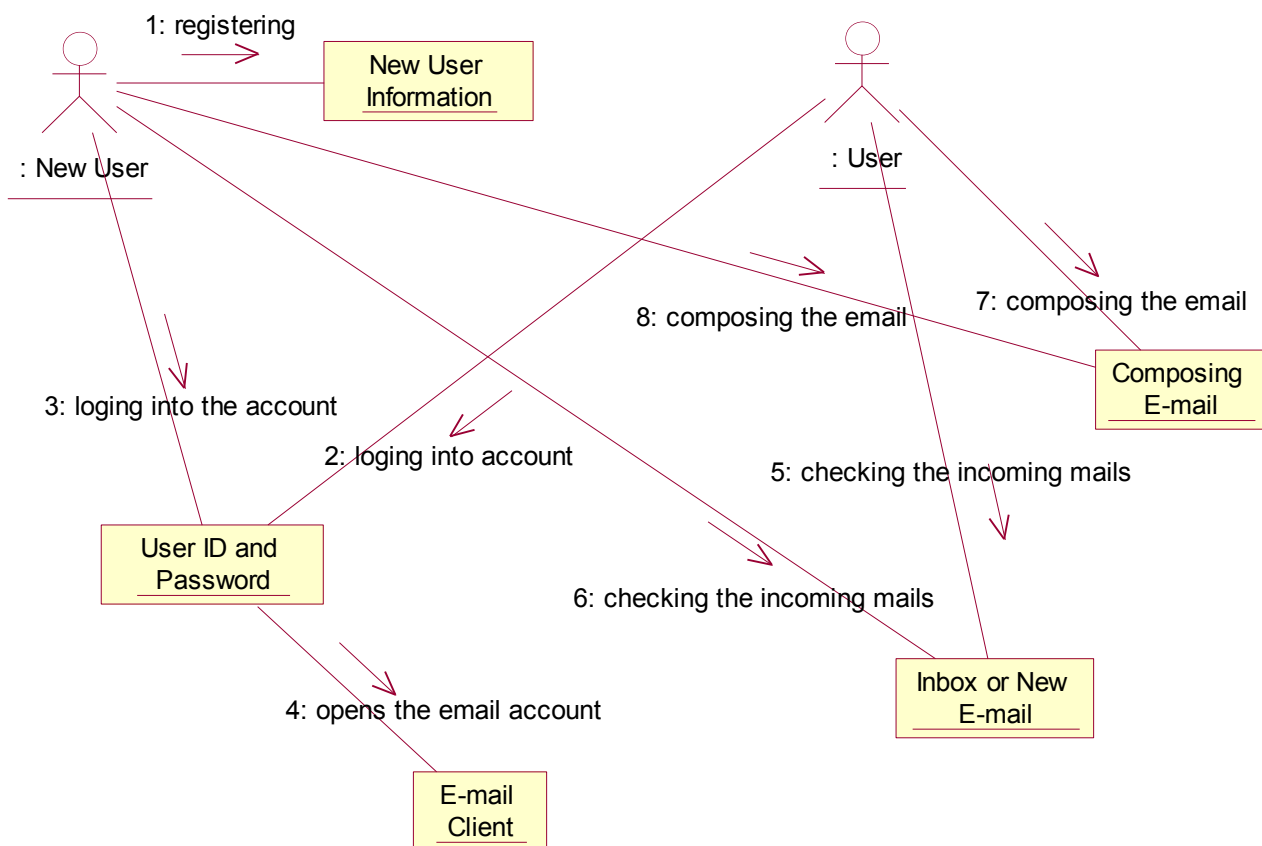


SMTP Sever:



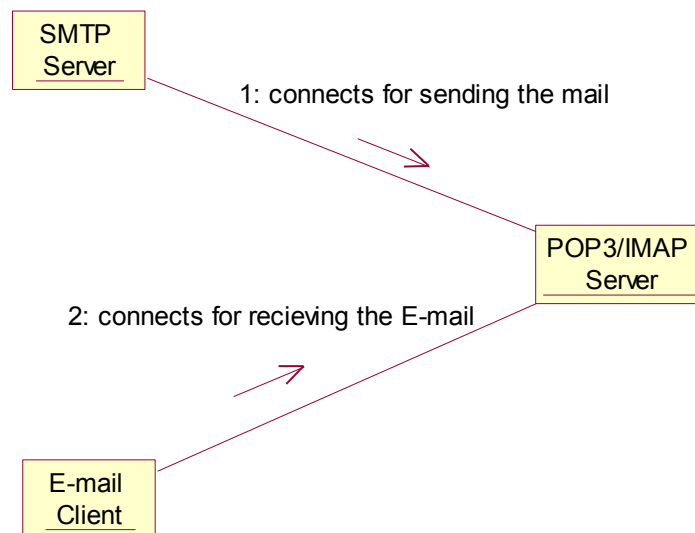
COLLABORATION DIAGRAM

E-mail Client/Logging into the E-mail Client:

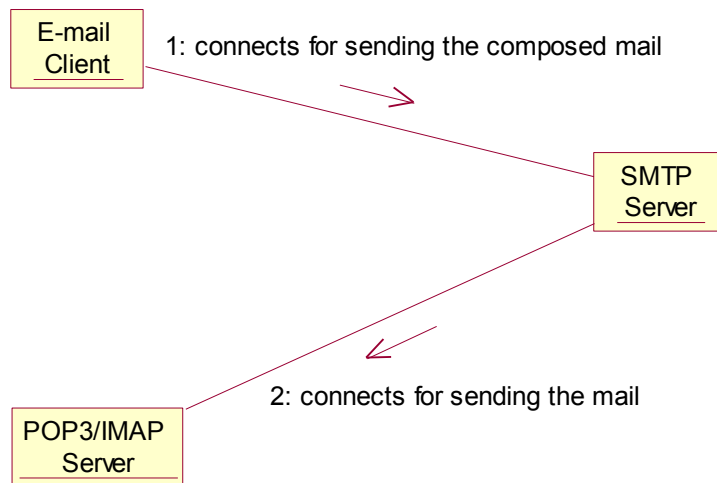


COLLABORATION DIAGRAM

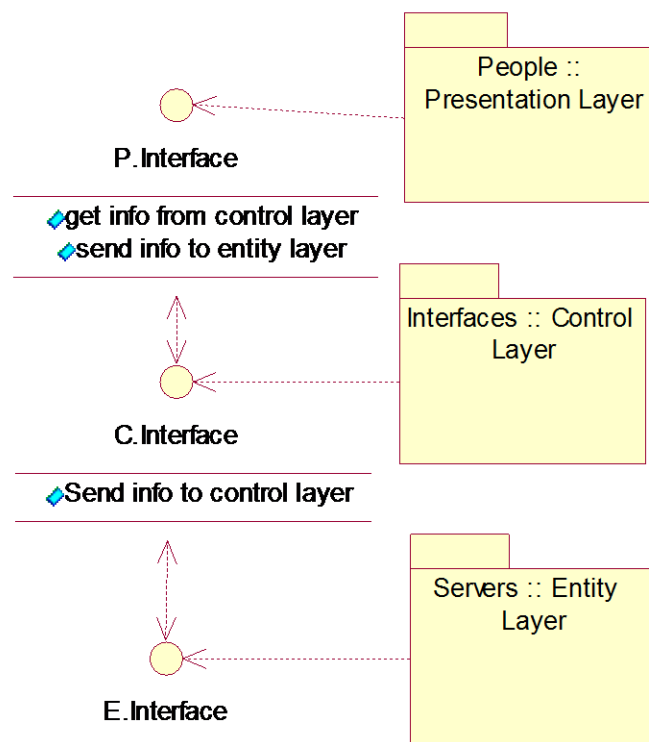
POP3/IMAP Server:



SMTP Sever:

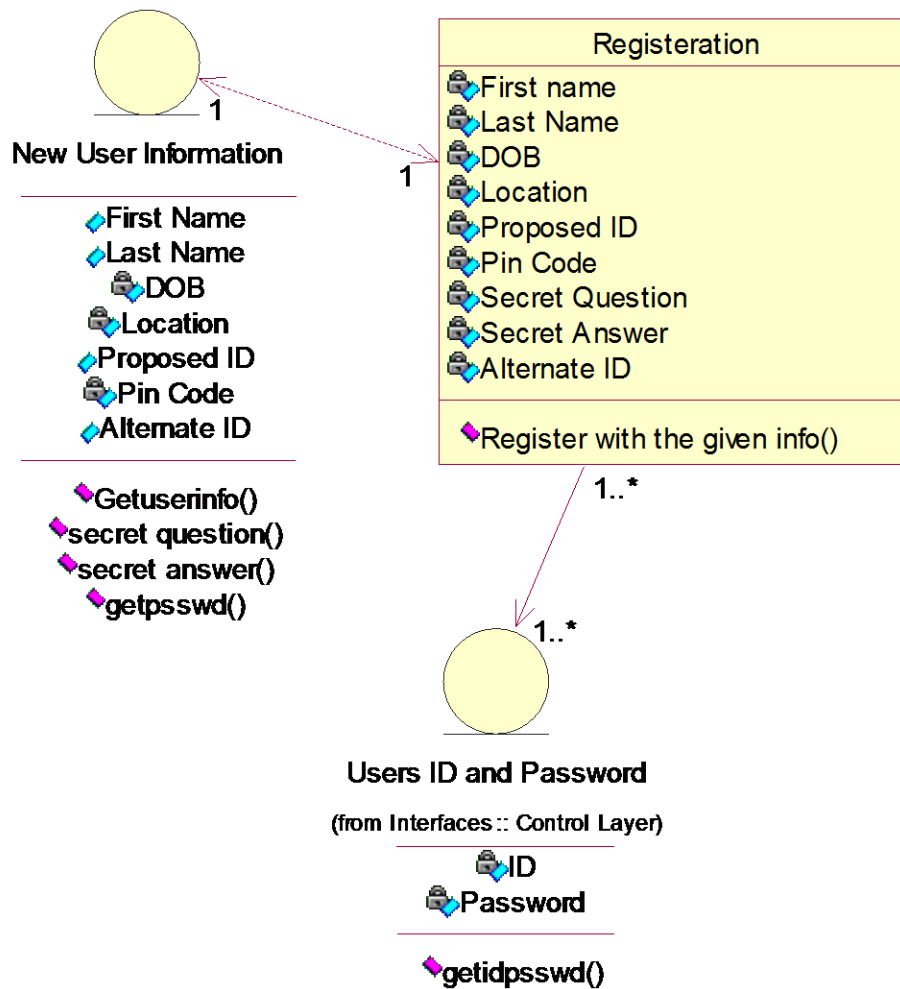


CLASS DIAGRAM



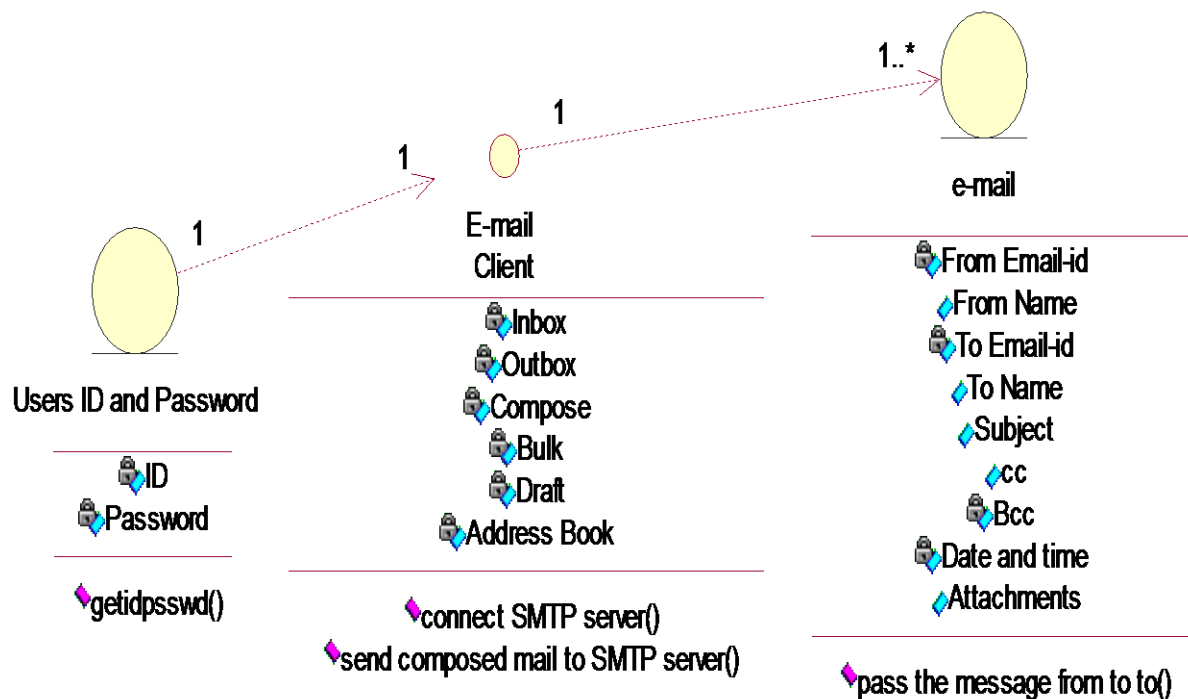
CLASS DIAGRAM

1: PEOPLE: PRESENTATION LAYER:

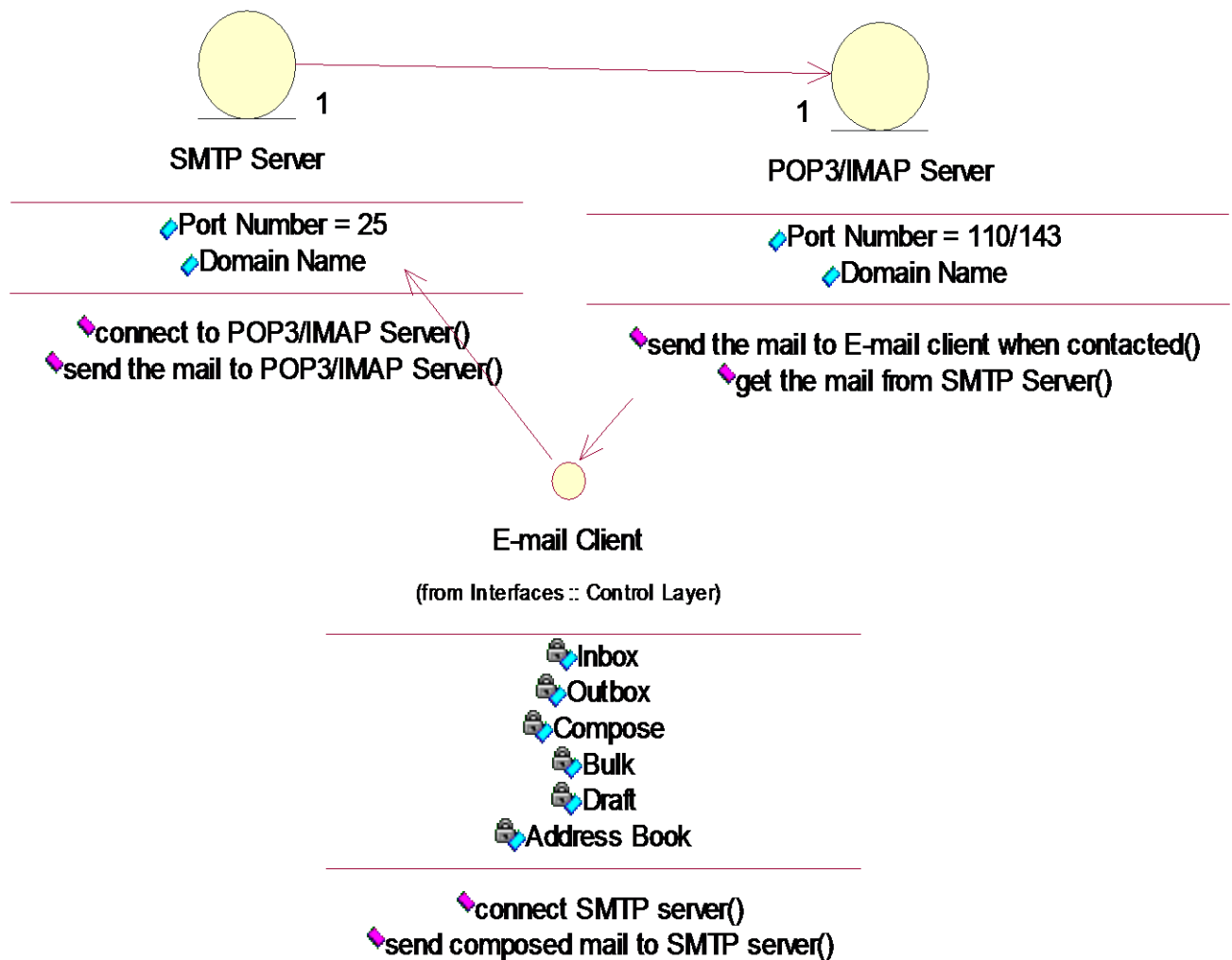


CLASS DIAGRAM

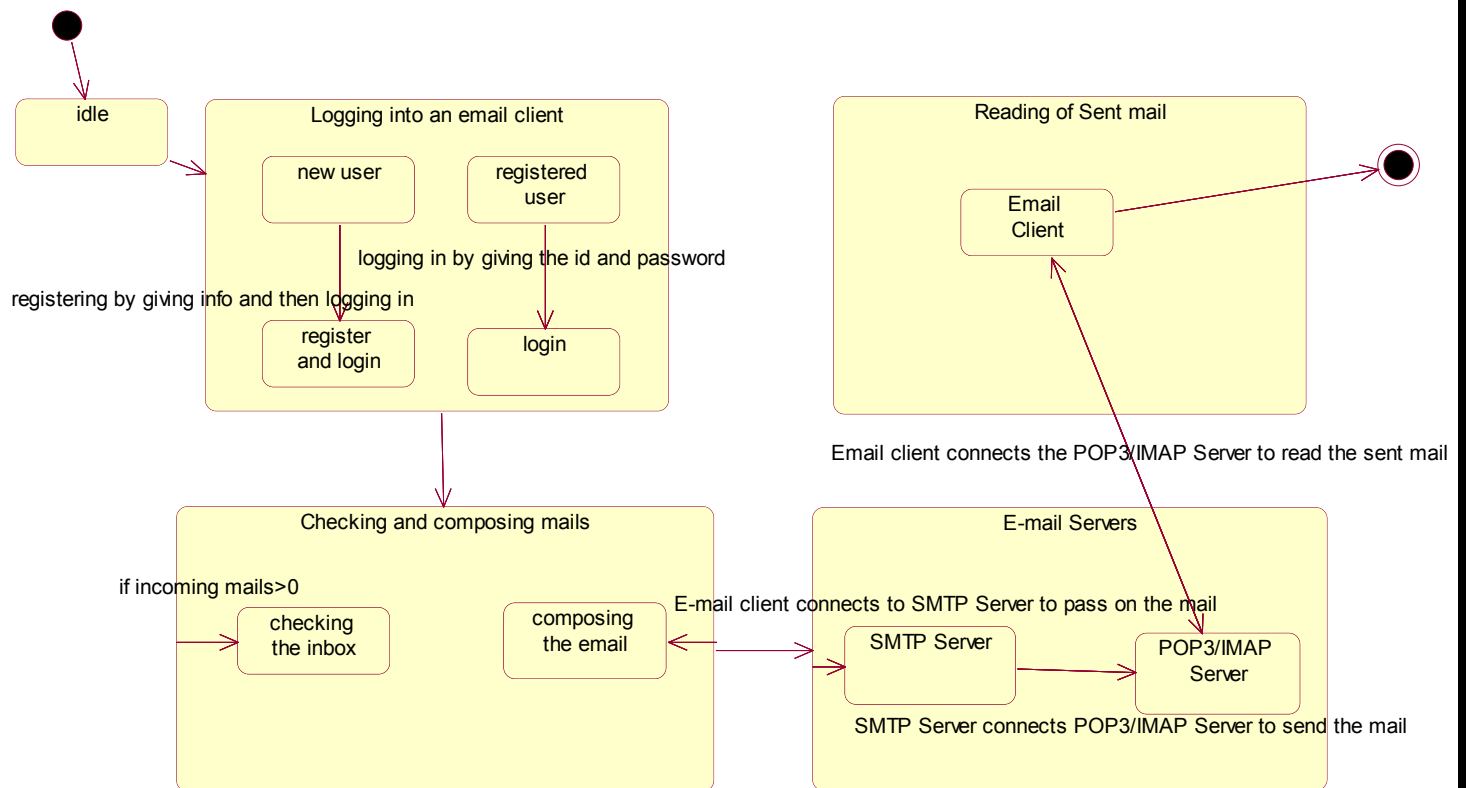
2: INTERFACES: CONTROL LAYER:



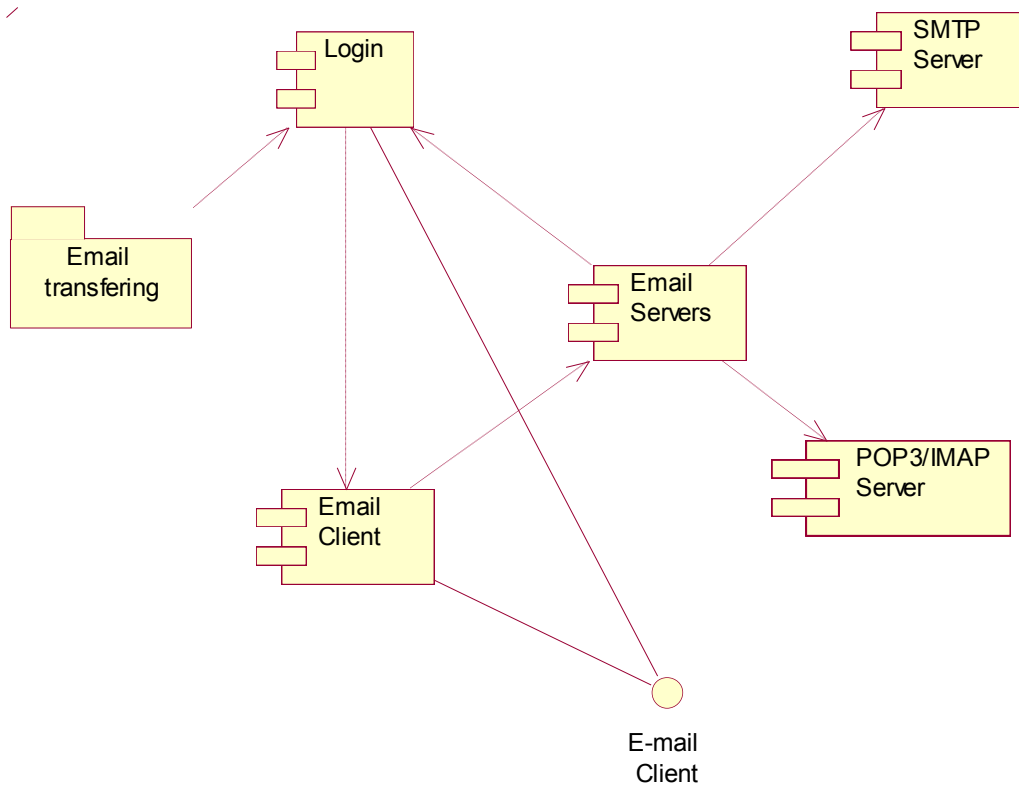
3: SERVERS: ENTITY LAYER:



STATE CHART DIAGRAM



COMPONENT DIAGRAM



DEPLOYMENT DIAGRAM

